

Python под нагрузкой: измерительное исследование JIT, рантайма, нативного кода и оптимизаций уровня сборки

Иванков Т. С.

Университет ИТМО

Научный руководитель: Стручков И. В., канд. техн. наук

Университет ИТМО

Аннотация

Решение о том, как ускорять программу на Python, означает выбор из дюжины приёмов, опубликованные результаты которых получены в несопоставимых условиях: разные машины, разные размеры входа, разные базовые линии — и почти никогда указание на то, чего приём стоит на коде, который он не ускоряет. Эта работа измеряет весь этот арсенал на одной машине, по одному протоколу, с одним происхождением интерпретаторов. Шесть ядер — три вычислительных, три ветвистых и с интенсивным выделением памяти — реализованы на чистом Python, Cython, Numba, Codon (с опережающей компиляцией и через его JIT), C (через ctypes) и Rust, а три вычислительных ядра дополнительно на NumPy и через привязку `rybind11`. CPython 3.10–3.14 и `free-threaded` сборка собраны здесь из релизных архивов одним компилятором и одной строкой `configure`. Семь конфигураций сборки 3.14.6 получены из одного дерева исходников, включая PGO с `full LTO` и экспериментальный `tier-2 JIT`. Релизный бинарик PyPy 7.3.20, а также форк Cinder от Meta с его расширениями CinderX, собранными из исходников на том же хосте, измерены относительно той же базовой линии. Всё хронометрирование выполняет `ryperf`, поэтому каждая цифра — медиана по значениям из многих независимых процессов, если в тексте не сказано иное, а там, где вывод держится на совпадении двух конфигураций, он несёт критерий значимости, а не пару округлённых медиан.

Шесть результатов стоит назвать сразу. На вычислительных циклах выбор компилируемого пути значит гораздо меньше, чем решение компилировать вообще: на скалярной редукции пути совпадают с точностью до 4.4%, а расходятся на 21% на сетке мандельброта и в 2.8 раза на наивном произведении матриц — в обе стороны и без постоянного победителя, — против тех 31–128×, которых стоит сама компиляция, тогда как результат 32× от результата 77× на одном и том же ядре отделяет контракт арифметики с плавающей точкой, а не инструмент. Этот контракт лишь отчасти является флагом компилятора: дизассемблирование каждого варианта флагов показывает, что `-O2` и `-O3` порождают одинаковый машинный код на двух ядрах из трёх и неразличимое время на третьем, тогда как `-march=native` меняет код всех трёх и время работы — в обе стороны: в 1.25 раза быстрее на сетке мандельброта и в 1.80 раза медленнее на произведении матриц. С редукцией `-ffast-math` разрывает последовательную цепочку зависимости по накопителю, и разорвать её можно прямо в исходнике, без всякого флага. На нерегулярном коде компилятор ограничивает не ветвление, а выделение памяти: C и Rust опережают Cython и Numba в 3.7–5.3 раза именно там, где создаются мелкие объекты, не более чем на 17% на насыщенном ветвлениями разборе байтов и вовсе не опережают на обходе графа. Прогресс интерпретатора реален, но распределён неравномерно: 3.14 выполняет операции с атрибутами и методами в 2.0–3.0 раза быстрее, чем 3.10, большая часть этого пришла с 3.11, а 3.12 часть отдаёт назад, оказываясь медленнее 3.11 на 14 из наших 16 операций. Отказ от GIL даёт 5.6–6.1× на восьми потоках ценой 1.05–1.14× однопоточного замедления, которое вносит сборка, а не блокировка, а код, горячий цикл которого уже нативный, масштабируется столь же хорошо и без этого (7.2×). Композиция направляется профилем, а не складывается: на смешанном конвейере единственное изменение, совпавшее с профилем стадий, дало 1.35×, тогда как наложение всех четырёх техник дало в лучшем случае 1.26× и 0.73× под GIL.

Ещё два результата относятся к системному уровню, на котором работает всё остальное. Сборка интерпретатора с одним только PGO стоит 1.02×, выигрыш — в комбинации: PGO вместе с `full LTO` даёт 1.10×, а добавление к этому `-march=native` — 1.12×, и единственная операция, которая проигрывает при всех них, — запись атрибута. Собственный экспериментальный JIT CPython даёт 1.04× в целом, улучшая 10 из наших 16 операций и замедляя 6. Со стороны альтернативных рантаймов: JIT из CinderX исполняет шесть наших нетипизированных ядер за 0.33–0.79× обычной сборки, если позволить ему компилировать после того, как интерпретатор специализировал байткод. Скомпилированные до первого вызова, те же ядра дают 0.85–1.53×, так что знак этого результата есть свойство методики. Его Static Python добавляет сверху ещё 11.4× (7.5× при медленном размещении кода) на типизированном ядре — хотя примитивные типы стоят того лишь потому, что их кто-то компилирует: в интерпретации, даже после того как мы дописали вышедший недоделанным специализирующий путь, они в 5.8 раза медленнее заменяемого ими кода с объектами. Наибольший множитель, доступный без правки исходника, даёт PyPy: он исполняет наши неизменные ядра на чистом Python в 3.2–42.8 раза быстрее, но единственный его вызов `ctypes`, передающий через границу двухмегабайтный буфер, стоит в 5.3 раза дороже, чем в CPython, тогда как вызовы, передающие только целые числа, укладываются в 7%: «перейти на другой рантайм» и «вынести горячий цикл в C» исключают друг друга лишь там, где через

границу идут объёмные данные. Весь код, все файлы результатов `ruperf` и генераторы графиков и таблиц входят в прилагаемый артефакт, каждый график и каждая таблица этой работы воспроизводятся из этих файлов командами генераторов из §14.

1 Введение

У команды, чья программа на Python работает слишком медленно, есть с десяток средств на выбор, и различаются они не столько обещаниями, сколько тем, чего требуют взамен. Одним хватает другого исполняемого файла: PyPy, экспериментальный JIT, появившийся внутри самого CPython с 3.13, более свежий выпуск или тот же выпуск, собранный с другими флагами (PGO, LTO, архитектурная настройка). Другим нужны аннотации и шаг сборки: Cython, Numba, Codon. Третьим — второй язык за FFI: C, C++, Rust. Четвёртым — другой рантайм и сопровождение, которое к нему прилагается: Cinder от Meta с расширениями CinderX [3], где есть JIT, режим статической типизации, параллельный сборщик и ленивые импорты. Ошибка в этом выборе дорого обходится в инженерном времени, а сузить его по разрозненным числам трудно.

Сами варианты хорошо известны. Труднее сопоставить их напрямую: числа для каждого получены на своём ядре, на неназванной машине и относительно неназванной базовой линии, не указывая, что именно фиксировалось, а значит, между собой не складываются. Вопросы же, которые на практике и решают исход — насколько большим должен быть вход, чтобы компилятор окупился, чего приём стоит в остальном коде, складываются ли два приёма, — обычно так и остаются без ответа.

Эта работа измеряет тот же набор средств в условиях, описанных в §3. Мы стремились получить таблицу, индексированную не инструментом, а тем, *что делает нагрузка*, где в клетках стоит *что помогло, насколько и чего это стоило*: вычислительные циклы, ветвистый код и работа с указателями, объектно-ориентированные нагрузки с интенсивным выделением памяти, смешанные конвейеры и системный уровень под всем этим. Таблица 8 и есть этот результат, и каждая её строка получена прилагаемым артефактом. Вклад работы:

1. Измерительная постановка (§3), в которой всё хронометрирование выполняет `ruperf` и каждое утверждение о совпадении двух конфигураций несёт критерий значимости, вместе с проверкой корректности: реализация измеряется только после того, как воспроизвела референсный результат на полном размере задачи.
2. Восемь измеренных вопросов (§4–§11), каждый с кодом, бенчмарком и графиком и каждый с практическим правилом в конце.
3. Кросс-технологическое сравнение на шести ядрах (три вычислительных, три ветвистых/с аллокациями) в семи реализациях, которое разделяет эффект *компиляции* и эффект *представления данных* и помещает границу между двумя классами на выделение памяти, а не на ветвление.
4. Отчёт на уровне машинного кода о том, что флаги компилятора меняют на самом деле (§4.1): дизассемблированный код каждого варианта флагов хешируется, из чего видно, что большинство обычно рекомендуемых флагов порождают для этих ядер побайтово одинаковый код, а единственный эффект ценой в несколько раз принадлежит цепочке зависимости, а не флагу, — что показано его воспроизведением в исходнике при неизменных флагах.
5. Отчёт о зрелости форка Cinder и CinderX, основанный на измерениях: сборка из исходников с теми же флагами `configure`, что у обычного CPython 3.14.6, в *обеих* конфигурациях сборки CinderX, с покрытием JIT, Static Python, ленивых импортов, параллельного сборщика мусора и облегчённых фреймов (§7). Адаптивную конфигурацию пришлось сперва доделать, иначе она не запускалась вовсе; чего именно ей не хватало, разобрано в §7. Недостающая половина лежит в артефакте и проверена собственным набором CinderX из 3910 статических тестов.
6. Артефакт (§14), в котором каждый график и каждая таблица работы восстанавливается из файлов результатов `ruperf`.

2 Связанные работы и измеряемые вопросы

Сравнительные работы по производительности реализаций Python обычно относятся к одному из трёх видов. (i) Отчёты по наборам бенчмарков для одной реализации: результаты `ruperformance` [2], сопровождающие каждый выпуск CPython, и сводки о скорости, которые проект PyPy ведёт по собственным выпускам [14]. Они строгие и продолжительны во времени, но сравнивают реализацию с ней же самой, а не одну стратегию ускорения с другой. (ii) Документация отдельного ускорителя — Numba, Cython, Codon, — которая по своей природе демонстрирует те ядра, на которых инструмент хорош. (iii) Отдельные замеры одного ядра двумя-тремя способами — полезный источник гипотез; чтобы такие числа складывались друг с другом, к ним нужны платформа, размер входа, протокол и цена приёма.

Таблица 1: Хост измерений. Каждое поле считано из артефакта, который произвёл прогон: из описания машины, записанного на самом хосте, и из метаданных `pyperf` по каждому прогону.

свойство	значение
CPU	Intel Core Ultra 7 255H
ядра	6 производительных + 10 энергоэффективных ядер
память	30.8 ГБ
операционная система	Ubuntu 24.04.4 LTS, Linux 7.0.0-28-generic (x86_64)
управление частотой	governor performance, турбо выключено
рандомизация адресов	полная
привязка бенчмарков к ядрам	процессоры 0-5 (класс производительных ядер)

Исследование, соединяющее все эти свойства сразу, встречается редко, и задача данной работы — соединить их в одном месте: (а) фиксировать нагрузку, меняя технологию ускорения, (б) фиксировать технологию, меняя рантайм, (в) приводить для того и другого одну и ту же статистику и (г) измерять цену каждого приёма рядом с его выигрышем. Инструментарий измерения взят стандартный там, где стандартный существует: всё хронометрирование выполняет `pyperf` [1], тот же прогонщик бенчмарков, которым пользуется работа над производительностью самого CPython, поэтому протокол ниже — его, а не наш собственный.

Восемь вопросов следуют классам нагрузок из §1. Каждый раздел формулирует свой вопрос, описывает эксперимент и заканчивается тем, что подтверждают числа.

- B1** Сколько на самом деле дают JIT-компиляция и нативный код на вычислительных ядрах, и в чём выигрыш — в языке, в компиляторе или в свободе, которую компилятору дают (B1б)? Чего стоит сама компиляция, прежде чем окупится (B1в)?
- B2** Ветвистый код и работа с указателями сопротивляются компиляции. Какое их свойство за это отвечает и сколько даёт вынос такого кода из Python?
- B3** Что дало объектно-нагруженному коду развитие самого интерпретатора (3.10 → 3.14: специализация байт-кода, бессмертные объекты, работа над аллокатором)?
- B4** Ленивые импорты, облегчённые фреймы, параллельный сборщик, JIT и статическая типизация в Cinder и CinderX: что из этого окупается и какой ценой?
- B5** Масштабирование без GIL и однопоточная цена сборки, которая его снимает.
- B6** Интерпретатор как программа на C: чего стоят PGO, LTO и архитектурные флаги на одном дереве исходников и чего стоит его собственный tier-2 JIT (B6б).
- B7** Композиция на реалистичном смешанном конвейере.
- B8** PyPy на тех же ядрах.

3 Методика

Машина. Она одна, описана в таблице 1, и каждое число этой работы получено на ней — включая измерения Cinder и CinderX из §7, которым нужен Linux-тулчейн и которые получают его здесь нативно. Сопоставимыми друг с другом, а не только внутри разделов, результаты работы делает именно то, что хост один.

Два её свойства — решения об измерении, а не описание, а третье унаследовано от настроенного состояния самого `pyperf`. Во-первых, регулятор частоты зафиксирован в `performance`, а турбо *выключено*: турбо-частота по построению непостоянна — она зависит от температуры, от того, сколько ядер занято, и от того, как долго длится нагрузка, — а работа, среди выводов которой есть «эти две конфигурации неразличимы», не может стоять на плывущей частоте. Ценой является абсолютная скорость, и она заплачена намеренно. Во-вторых, у процессора ядра двух классов производительности, поэтому каждый однопоточный бенчмарк привязан к производительному классу, без этого один и тот же бенчмарк может быть измерен на одном классе в одном рабочем процессе и на другом — в следующем, и разница уйдёт в разброс. Исследование масштабирования по потокам из §8 — единственное исключение, поскольку оно перебирает больше потоков, чем у этого класса ядер. В-третьих, рандомизация адресного пространства оставлена *включённой*, как того и требует настройка `pyperf`: довод в пользу измерения по многим процессам состоит в том, что размещение, состояние аллокатора и зерно хеша между ними различаются, а отключение рандомизации убрало бы ровно ту дисперсию, на которой этот довод держится. Таблица 1 приводит все три свойства такими, какими их наблюдал `pyperf` во время прогона, а не такими, какими мы их задумывали.

Протокол измерения. Каждая измеренная цифра этой работы получена с помощью `pyperf` [1] — той же системы запуска бенчмарков, которой пользуется работа над производительностью самого CPython. Мы берём его протокол как есть: для каждого бенчмарка он калибрует счётчик внутреннего цикла, пока одно значение не займёт не менее 100 мс, а затем запускает **двадцать независимых рабочих процессов**, каждый из которых отбрасывает разогревающее значение и сохраняет три, — итого шестьдесят значений на бенчмарк. Двадцать сразу по карману не всякому набору: там, где одно значение — ядро длиной от миллисекунд до секунд, процессов десять. Вычислительный набор, набор флагов и ветвистый набор прогоняются дважды, с обратным порядком бенчмарков во втором проходе, и оба прохода дописываются в один файл — тридцать значений на проход, шестьдесят в файле, из двадцати процессов, и заодно прямая проверка того, что проходы согласуются; наборы `CinderX`, `Static Python`, масштабирования сборщика, конвейера и точки окупаемости идут одним проходом на десяти процессах: тридцать значений. Масштабирование по потокам, где одно значение есть целый многосекундный прогон, идёт одним проходом на пяти процессах: пятнадцать значений. На интерпретаторе, который сообщает о наличии JIT — это `PyPy` и сборка `jit`, — `pyperf` сам переходит на шесть процессов по десять значений, и мы ему это оставляем. Число процессов записывается вместе с каждым результатом, и важно здесь то, что это всегда несколько отдельных процессов, а никогда не один. Повторение на уровне процессов — это и есть та часть, которая важна, и та, которую самописный таймер внутри процесса дать не может: размещение адресного пространства, состояние аллокатора, зерно хеша и размещение кода различаются от процесса к процессу, а показатель разброса, посчитанный внутри одного процесса, не включает ничего из этого. Мы приводим медиану и относительное медианное абсолютное отклонение (MAD).

`pyperf` применяет и собственный критерий стабильности — 95% уверенности в разбросе менее 1% — и предупреждает, когда бенчмарк его не проходит. Критерий написан для настроенной Linux-машины, а этот хост не может держать частоту настолько ровно. По 69 бенчмаркам вычислительного набора относительный MAD составляет 0.2% в медиане и 4.8% в худшем случае, и предупреждение несёт примерно пятая часть. Вместо того чтобы объявлять критерий, которого мы не выполнили, мы указываем разрешающую способность: различия больше примерно 10% этими измерениями разрешаются, различия в несколько процентов парой медиан не разрешаются, и там, где работе нужно такое различие, она пользуется критерием значимости.

Два вида измерений требуют большего, чем просто вызываемая функция. Нагрузки, которые потребляют или изменяют свой вход — пауза сборки мусора, стадия конвейера, — регистрируются через `bench_time_func`, где восстановление входа происходит внутри цикла, но вне измеряемой области, так что подготовка никогда не идёт в счёт. Стоимости уровня процесса — запуск интерпретатора, задержка импорта — идут через `bench_command`, который измеряет сам процесс, потому что таймер внутри процесса не может наблюдать ту часть запуска, которая ему предшествует.

Там, где работа говорит, что две конфигурации не различаются, это критерий значимости `pyperf` (критерий Стьюдента на уровне 95%) по двум наборам значений, а не разглядывание двух округлённых чисел. Исследование специализации из §6 идёт с **фиксированным** счётчиком цикла 200 вместо калиброванного: оно измеряет один вызов функции длиной в несколько микросекунд, и достижение цели `pyperf` в 100 мс означало бы десятки тысяч свежих компиляций внутри одного значения. Одно значение там — примерно от 75 мс на 3.10 до 103 мс на 3.14, из которых примерно 1% приходится на измеряемые вызовы, поэтому те числа несут больше относительного шума, чем остальная работа, и читаются как кривая, а не как отдельные точки.

Проверка корректности. Ядра определены один раз на Python (`kernels_py.py`, `branchy_py.py`) — любая другая реализация обязана воспроизвести референсное значение до того, как будет измерена. Несовпадение записывается как результат (`wrong_result`), а не как время. Единственное исключение — переассоциация операций с плавающей точкой в духе `fastmath`: ей разрешена относительная погрешность 10^{-3} , а наблюдаемое отклонение сохраняется рядом с замером. §4.1 приводит его явно.

Что фиксируется. Внутри одного вопроса меняется ровно один фактор: технология реализации (§4, §5), набор флагов компилятора при неизменных исходниках на C (§4.1), версия интерпретатора (§6), рантайм и его функции (§7), конфигурация GIL (§8) или флаги сборки самого интерпретатора (§9). Раскладка данных считается частью реализации: чистый Python работает со списками, компилируемые и нативные реализации — с непрерывными буферами `float64`, ровно как поступил бы инженер, который их внедряет. Там, где важна цена преобразования между ними, она измеряется и приводится отдельно (§5, §10).

Дрейф машины. Машина, которая движется, пока идёт измерение, превращает прошедшее время в кажущийся эффект — поэтому протокол выше и не подлежит обсуждению: измерение, которое берёт все свои выборки подряд, одно от другого отличить не может, и ровно такой результат мы получили в первом проходе — он записан ниже, в перечне того, что поймал протокол. Стоит быть точным в том, от чего устройство `pyperf` здесь защищает, а от чего нет, потому что эти две вещи легко смешать. Рабочие процессы убирают всё, что меняется *от процесса к процессу* — четыре свойства, перечисленные выше в описании протокола, — и это как раз то

искажение, которого однопроцессный таймер даже не видит. Они *не* размазывают бенчмарк по прогону: `pyperf` доводит до конца все процессы одного бенчмарка, прежде чем начать следующий, поэтому два бенчмарка, сравниваемые через длинный прогон, по-прежнему разнесены во времени, и замедляющаяся машина покажет это разнесение как эффект.

Оставшуюся часть закрывают два средства. Каждый набор, кроме наборов CinderX, Static Python и отдельного прогона Codon с опережающей компиляцией, регистрирует *машинную пробу* — фиксированный нативный вычислительный цикл, вызываемый через `stypes`, стоимость которого не зависит от интерпретатора, — измеряемую `pyperf` как любой другой бенчмарк, так что сравнение её между двумя прогонами есть прямая проверка того, изменилась ли машина, с тем же критерием значимости, что и всё остальное. А там, где сравнение держится на нескольких процентах, две конфигурации измеряются *попеременно*, а не одна за другой: короткие запуски с меняющимся каждый раунд порядком, дописываемые каждый в свой файл (`bench/b1_compute/ab_flags.sh`), так что обе выбирают один и тот же отрезок времени. Этот протокол существует, но ни одно число в работе из него не получено: контроль дрейфа в §4.1 — это двухпроходный план, описанный выше.

Статистика и шум. Медианы с относительным MAD, а не средние со стандартным отклонением, потому что распределения скошены вправо шумом планировщика. Каждый прогон сделан на простаивающей машине, подключённой к сети, без посторонней нагрузки — и это работа научилась проверять, а не предполагать: чего это стоило, записано в §13. Абсолютные значения специфичны для этой машины и её зафиксированной частоты с выключенным турбо, утверждения работы касаются отношений.

Как читать множители. Запись « $N\times$ » всюду означает *ускорение*: во сколько раз быстрее выполняется то, что стоит первым, — поэтому $N > 1$ читается как выигрыш, а $N < 1$ как проигрыш. Единственное исключение оговаривается на месте оборотом «за $N\times$ времени такой-то конфигурации»: там это доля времени, и меньшее число означает более быстрое исполнение. Цена приёма приводится как замедление («в 1.14 раза медленнее»), а не как доля пропускной способности, чтобы направление читалось из самой записи.

Что поймал протокол. Три конкретные ошибки нашёл протокол, а не чтение кода, — и это главный аргумент в пользу того, чтобы протокол вообще иметь. (i) Первый проход брал выборки подряд по конфигурациям и дал аккуратный, правдоподобный и *ложный* результат: ядра на чистом Python как будто монотонно замедлялись от 3.11 к 3.14 — исключительно как функция позиции в прогоне. (ii) Параллельные варианты конвейера из §10 выдали контрольную сумму, отличную от референсной и различавшуюся между прогонами, — это гонка за выходную ячейку `stypes`, разделяемую четырьмя потоками. Бенчмарк, измеряющий только время, доложил бы о варианте с гонкой как об ускорении, потому что потерянные обновления уменьшают объём работы, проверка корректности отвергла его до того, как он вообще был измерен. (iii) Наш первый замер разогрева специализации дал плоскую кривую на всех версиях, потому что код-объект компилировался один раз и переиспользовался между репликами, так что холодной была только первая. Компиляция на каждой реплике и замена тела цикла линейным кодом, чтобы каждая инструкция исполнялась один раз за вызов, сделали эффект видимым (§6).

4 В1: сколько дают JIT и нативный код на вычислительных ядрах?

Вопрос. Сколько даёт выход из интерпретатора на вычислительном ядре и насколько это зависит от того, каким именно способом из него выйти? Для одного и того же цикла инженеру доступны шесть технологий — опережающая компиляция, два JIT, два интерфейса к нативному коду и векторизованная библиотека, — и выбор между ними обычно делается без сопоставимых чисел хотя бы для одной из них.

Постановка. Три ядра: последовательная сумма 10^6 значений `float64` (канонический пример), сетка мандельброта 200×150 с 30 итерациями (вложенные циклы, плавающая точка, внутренний `while` с зависимостью от данных) и наивное произведение матриц 96×96 типа `float64` (три вложенных цикла, доступ с шагом). Реализации: цикл на чистом Python, встроенная `sum()` где применима, NumPy, Cython [9], Numba [8] (с `fastmath` и без), Codon [10] обоими доступными у него путями — модулем расширения, собранным заранее через `codon build -pyext`, и теми же ядрами под `@codon.jit`, — те же ядра на C через `stypes` и через модуль расширения `pybind11`, а также Rust через `stypes`. Все замеры на CPython 3.14.6, совпадение результатов обязательно до измерения. Расширения на C, C++ и Cython собраны одним и тем же `clang 18.1.3 -O3 -march=native, a cdylib` на Rust — `c target-cpu=native`: это рецепт по умолчанию, и §4.1 показывает, что на одном из этих ядер он не нейтрален.

Результат: компилируемые пути совпадают, а библиотека и флаг — нет. Таблица 2 и рисунок 1 дают измерение. На ядре суммирования — сумма 10^6 значений `float64` — цикл на чистом Python занимает 32.23 мс,

Таблица 2: B1: вычислительные ядра на CPython 3.14.6, на единственном хосте из таблицы 1. Медианное время вызова и ускорение относительно цикла на чистом Python. Строка `matmul` для NumPy — это данные о поставляемом BLAS, а не о компиляторе.

реализация	сумма массива (10^6 f64)		мандельброт 200×150		matmul 96×96	
	мс	×	мс	×	мс	×
цикл на CPython	32.23	1.0	46.92	1.0	73.27	1.0
встроенная <code>sum()</code>	6.25	5.2	—	—	—	—
Cython	1.01	31.9	0.751	62.5	1.01	72.3
Codon AOT	1.01	31.9	0.865	54.3	1.50	49.0
Codon AOT (указатель)	1.01	31.9	—	—	0.587	124.7
Numba	1.01	31.9	0.864	54.3	0.571	128.4
Numba fastmath	0.419	76.9	0.683	68.7	0.535	137.0
Codon JIT	1.05	30.6	0.910	51.5	1.59	46.0
C (ctypes)	1.01	31.8	0.776	60.5	1.01	72.2
C (pybind11)	1.01	31.9	0.752	62.4	1.01	72.2
Rust (ctypes)	1.01	31.8	0.888	52.8	0.830	88.3
NumPy	0.411	78.4	9.98	4.7	0.049	1495.9

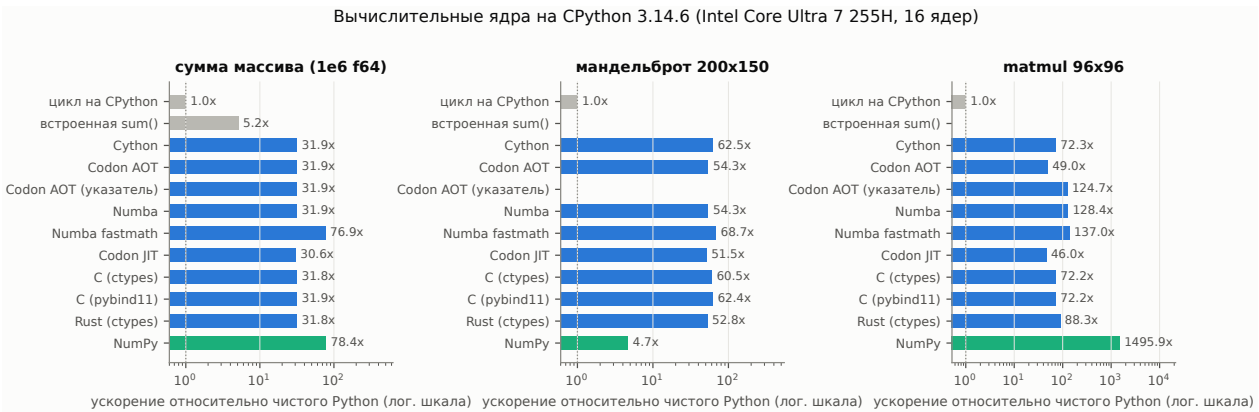


Рис. 1: B1: измеренные ускорения относительно цикла на чистом Python (лог. шкала). Каждый столбец — медиана по шестидесяти значениям `runner.f` из двадцати рабочих процессов.

встроенная `sum()` — 6.25 мс, то есть $5.2\times$ за единственное изменение, которому не нужен шаг сборки, а каждая статически компилируемая реализация попадает в полосу шириной 0.31 %: Cython $31.9\times$, Numba $31.9\times$, C через `ctypes` $31.8\times$, C через `pybind11` $31.9\times$, Rust $31.8\times$, Codon с опережающей компиляцией $31.9\times$ и он же с указателями $31.9\times$ (от 1.009 до 1.012 мс). Внутри такой узкой полосы критерий значимости одни пары разрешает, а другие нет: Numba против C при $t = -3.79$ и две границы к C между собой при $t = 3.30$, но Cython против C при $t = -1.47$ и против Rust при $t = -0.25$, — а десятая доля процента лежит ниже разрешающей способности, указанной в §3. Это ядро компилируемые пути не упорядочивает. Оно показывает отрицательный результат вместе с причиной: все эти реализации компилируют одну и ту же последовательную цепочку сложений, где каждое ждёт предыдущего, поэтому язык не решает ничего. Решает разрешение эту цепочку разорвать — строки с векторизацией и `fast-math` ниже уходят от этой полосы в 2.5 раза на том же ядре. Группировка держится слабее на сетке мандельброта ($52.8\text{--}62.5\times$) и распадается на произведении матриц ($49.0\text{--}128.4\times$) по причине, которая принадлежит компилятору и разбирается ниже. На сумме массива критерий разводит и две границы к C — на одну десятую процента; единственное же место, где они расходятся на заметную величину, — мандельброт, на 3.3 % ($t = 48.3$): один исходник, одни флаги, два пути компоновки и полезное напоминание о том, насколько крупным может измериться «одна и та же программа дважды».

Интереснее самой заголовочной цифры вот что.

Во-первых, **плато задаёт не язык**. Numba с `fastmath` достигает $76.9\times$ на суммировании — в 2.41 раза дальше компилированного плато, — и тот же исходник на C, собранный с `-ffast-math`, даёт 0.418 мс, то есть $77.1\times$ (§4.1). Базовая линия на чистом Python берётся при этом из набора настоящего раздела, поскольку в наборе флагов её нет: это единственное в работе отношение, снятое между двумя разными наборами. Ничего в языке не изменилось, изменилось разрешение переассоциировать сложения с плавающей точкой, что разрывает последовательную цепочку зависимостей по накопителю. §4.1 показывает, что множитель принадлежит этой цепочке, а не какому-либо компилятору, — воспроизводя его при неизменных флагах.

Во-вторых, **там, где есть библиотека, она побеждает компилятор**. NumPy даёт $78.4\times$ на суммировании

(это само по себе векторизованное ядро с попарным суммированием) и $1495.9\times$ на произведении матриц, где вызов уходит в поставляемый BLAS — на три порядка дальше наивного тройного цикла, который добросовестно воспроизводят все остальные реализации. И наоборот, на мандельброте NumPy — *самый медленный* из ускоренных вариантов ($4.7\times$), потому что цикл escape-time с досрочным выходом по каждому пикселю приходится выражать проходами по всей сетке с маской. Поэтому одна и та же технология даёт $1495.9\times$ на одном ядре и $4.7\times$ на другом, одно утверждение «NumPy быстрый» не несёт информации.

NumPy — к тому же единственная реализация в этом сравнении, которая не однопоточна: её произведение матриц уходит в BLAS, забирающий столько потоков, сколько позволяет маска привязки, тогда как Cython, Numba, C и Rust получают одно ядро. Оставить это неоговорённым значило бы сравнивать несколько ядер векторизованного BLAS с одним ядром всего остального, поэтому набор измеряет вторую строку NumPy, `numpy_1t`: тот же код, но `threadpoolctl` держит BLAS в одном потоке вне измеряемой области. Две строки статистически неразличимы на двух ядрах из трёх: сумма массива 0.4111 против 0.4111 мс ($1.0001\times$ при $t = 0.49$) и произведение матриц 0.0490 против 0.0488 мс ($0.9966\times$ при $t = 1.66$) — оба различия незначимы. Исключение одно — мандельброт, 9.98 против 9.96 мс ($0.9982\times$, $t = 2.07$): критерий его разрешает, и составляет оно две десятых процента. На этих размерах задачи многопоточность не даёт ничего измеримого, так что обычная строка `numpy` является сравнением на равных условиях.

В-третьих, Numba даёт $128.4\times$ на произведении матриц против 72.2 – $72.3\times$ у Cython/C/pybind11 и $88.3\times$ у Rust. Это единственное место в B1, где компилируемые пути расходятся всерьёз, и причина принадлежит не технологии, а тому, чем собраны три отстающие строки: `-O3 -march=native`. На этом ядре данный флаг — регрессия, которую измеряет §4.1 и которая целиком объясняет разрыв. Кажущееся преимущество JIT здесь есть свойство рецепта сборки, а не компиляции на лету.

В-четвёртых, Codon — единственная здесь технология, чей результат зависит не столько от неё самой, сколько от того, как написано ядро. Две его строки на произведении матриц дают $49.0\times$ и $124.7\times$, то есть различаются в 2.55 раза, а исходный текст различается тремя строками: индексируется массив или взятый из него один раз указатель. Заявить в сигнатуре непрерывность массива, как это делает `double[:,1]` в Cython, Codon не умеет, поэтому индексация `ndarray` идёт через шаги, известные лишь во время исполнения, и каждый доступ к элементу тащит за собой это умножение со смещением; форма с указателем его убирает. Идиоматичная строка из-за этого проигрывает Cython ($72.3\times$), а строка с указателем обгоняет и его, и все нативные строки таблицы. Этот разброс шире, чем разница между любыми двумя языками здесь, — и в том всё дело: на данном ядре решал не выбор между Python и Rust, а одна строка внутри одной функции. Обе строки дают эталонный результат точно, так что проверка корректности между ними не различает.

Два пути Codon заодно дают цену самого JIT. Предварительная и внутрипроцессная компиляция одного и того же текста расходятся на 4–6 % (1.010 против 1.054 мс, 0.865 против 0.910 мс, 1.497 против 1.591 мс) — это цена моста, а не порождаемого кода. Разделяет их задержка: довести одно ядро до работы — от уже импортированного ускорителя до скомпилированной функции — стоит 1.33 с под JIT Codon против 215 мс под Numba. Доля Codon делится на 0.55 с декорирования функции и 0.79 с первого вызова с новой сигнатурой, ничего из этого на диске не кэшируется, а декорирование берётся с каждой функции, а не один раз. §4.2 переводит это в числа вызовов до окупаемости и отдельно приводит цену загрузки каждого ускорителя, где расхождение значительно больше. Предварительная сборка не платит ничего из этого, поэтому `-ruext` и является предпочтительным режимом, а сравнивать его следует с Cython.

Вывод. Уход из интерпретатора стоит одного-двух порядков на вычислительных ядрах. Каким именно путём из него выйти, значит гораздо меньше, а на скалярной редукции — почти ничего: восемь компилируемых реализаций внутри 4.4 %, а семь из них — внутри 0.31 %, причём про две из них доказано, что это одна и та же программа. Там, где пути всё-таки расходятся, они расходятся в 2.79 раза на произведении матриц — и ни в одном из двух случаев причиной не служит технология, названная в строке: с одного конца флаг компилятора, оказавшийся на этом ядре регрессией (§4.1), с другого — одна строка, решающая, сколько адресной арифметики несёт внутренний цикл. Поэтому выбор между Cython, Codon, Numba, C и Rust этими измерениями не решается, а то, чем он решается — сложность сборки, отлаживаемость, владение кодом — лежит за пределами измеряемого в этой работе. Про сам выбор измерения говорят одно: у Codon разброс относительно самого себя ($2.55\times$) шире, чем между любыми двумя остальными, и «какая технология» для него просто не та единица выбора. Уровнем ниже выбор, определяющий производительность, — это контракт арифметики с плавающей точкой, стоящий ещё $2.4\times$, а уровнем выше — вопрос «не делает ли это уже библиотека», стоящий до $1495.9\times$.

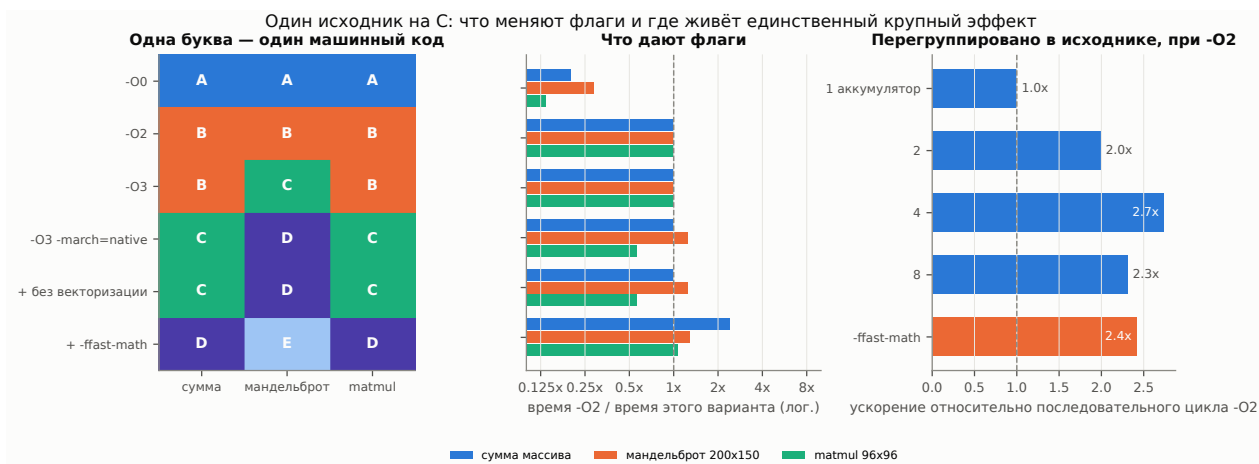


Рис. 2: В16: один исходник на C, шесть наборов флагов, сгруппированных по машинному коду, который они на самом деле порождают, плюс лестница накопителей.

Таблица 3: Какие флаги меняют порождаемый код. Варианты, соединённые знаком «=», — побайтово одинаковый машинный код для этого ядра, по тому же адресу в образе (results/codegen_identity.json).

ядро	различных программ среди шести наборов флагов
c_arraysum	-00 -02 = -03 native = без векторизатора -ffast-math
c_mandelbrot	-00 -02 -03 native = без векторизатора -ffast-math
c_matmul	-00 -02 = -03 native = без векторизатора -ffast-math

4.1 В16: выигрыш в языке или в компиляторе?

Вопрос. В1 обнаруживает, что выбор одной из четырёх компилируемых технологий стоит гораздо меньше, чем решение компилировать, а это ставит вопрос о том, что вообще вносит компилятор. Если выигрыш порождает генерация кода под целевой процессор, то один исходник на C, собранный с разными уровнями оптимизации и под разные архитектуры, должен расходиться так же. Расходится ли он, и если набор флагов не даёт разницы во времени, то это артефакт измерения или свойство компилятора?

Постановка. Один исходник на C (kernels_c.c) собирается шестью способами — -00, -02, -03, -03 -march=native, то же с отключённым векторизатором (-fno-vectorize -fno-slp-vectorize) и то же с -ffast-math, — и все шесть загружаются через ctypes в одном процессе, а Numba измеряется на тех же данных с fastmath и без него.

Прежде чем что-либо измерять, мы спрашиваем, что флаги на самом деле изменили. codegen_diff.sh дизассемблирует каждое ядро из каждой из шести разделяемых библиотек, нормализует адреса и цели переходов и хеширует поток инструкций, таблица 3 — результат. Два варианта с одинаковым хешем — это одна и та же программа, и любое различие в их измеренном времени тогда есть утверждение о машине, а не о флагах.

Результат: два из этих флагов — не переменные, а один — крупная переменная. Рисунок 2 сводит все три панели того, что следует ниже.

-03 здесь не переменная. На суммировании и на произведении матриц он порождает те же инструкции, что и -02, в том же порядке и по тому же адресу (38 и 92 инструкции соответственно), так что равное время выполнения — единственный возможный исход, на манделъброте это другая программа ровно на одну инструкцию (68 против 67) и всё равно не другое время ($0.9993 \times$, $t = 0.81$, незначимо). Причина видна в исходнике: -03 у clang отличается от -02 в основном порогами встраивания и разворачивания и несколькими преобразованиями циклов, ни одно из которых к трём самодостаточным циклам не применяется. Карта тождества даёт заодно и шкалу разрешения: на произведении матриц побайтово одинаковые сборки -02 и -03 измеряются как 0.5639 против 0.5633 мс — различие в 0.11 %, и ruperf называет его значимым ($t = 4.11$). Одна программа, две метки, значимое различие: вот чего стоит 0.1 % в этой измерительной обвязке, и это ограничивает то, как следует читать любое малое отношение в остальной работе.

Отключение векторизатора тоже не является переменной, и по причине, которую стоит назвать. Для всех трёх ядер сборка без векторизатора хешируется одинаково со сборкой -march=native, из которой она получена, и времена совпадают: $0.9999 \times$ при $t = 0.53$, $1.0009 \times$ при $t = -0.49$ и $0.9998 \times$ при $t = 0.69$, ни одно различие

не значимо. Дело не в том, что векторизация неважна, — дело в том, что clang вообще не породил ни одной векторной инструкции ни для одного из трёх ядер при `-O3 -march=native`, так что переключателю нечего убирать. Единственная сборка в этой работе, которая суммирует всё-таки векторизует, — сборка с `fast-math` (9 векторных операций на 41 инструкцию), и добирается она туда через разрешение переассоциировать, а не через просьбу векторизовать.

Архитектурный флаг переменной *является* — на всех трёх ядрах, и знак у него непостоянен. `-march=native` меняет порождаемый код везде: суммирование с 38 инструкций до 37, манделъброт с 67 до 121 (операций с плавающей точкой — с 17 до 32), произведение матриц с 92 до 96. Что это даёт, зависит от ядра: вовсе ничего на суммировании (1.0104 против 1.0106 мс, $t = 0.19$, незначимо), $1.25\times$ на манделъброте ($t = 410$, значимо) и $0.557\times$ на произведении матриц ($t = -646$, значимо): название точной модели процессора заставило это ядро идти в 1.80 раза дольше, чем при простом `-O2`, — 1.0124 против 0.5639 мс. Регрессия не гипотетическая: именно с `-O3 -march=native` собраны строки C, `pybind11` и `Cython` из §4, и она целиком составляет тот разрыв, который этот раздел измеряет между Numba и путями опережающей компиляции на произведении матриц.

`-O0` обходится дорого на каждом здешнем ядре: $5.0\times$ на суммировании, $3.5\times$ на манделъброте, $7.4\times$ на произведении матриц.

Результат: единственный значимый флаг — по сути не флаг. `-ffast-math` стоит $2.42\times$ на суммировании, $1.03\times$ на манделъброте и $1.92\times$ на произведении матриц — всё относительно сборки `-O3 -march=native`, к которой флаг и добавлен. Счётчики инструкций говорят, что он сделал на суммировании: сборка `-O2` несёт девять операций с плавающей точкой и ни одной векторной, а сборка с `fast-math` — десять с плавающей точкой и девять векторных, это единственный вариант этого ядра, который вообще векторизуется, и сделать это он может только после того, как переассоциация суммы разрешена. Значит, эффект — это разорванная цепочка зависимости, а не выбор инструкций, — и потому он должен воспроизводиться без флага, и он воспроизводится. `assum_c.c` выписывает ту же редукцию с 1, 2, 4 и 8 независимыми накопителями и собирается на обычном `-O2` вообще без `fast-math`:

накопителей в исходнике	мс	против одного накопителя
1	1.0101	1.00 $\times$
2	0.5065	1.99 $\times$
4	0.3689	2.74$\times$
8	0.4368	2.31 $\times$
<code>-O3 -march=native -ffast-math</code>	0.4182	2.42 $\times$

Исходник с одним накопителем — это последовательный цикл `-O2` под другим именем, и он так и измеряется ($1.0005\times$, $t = -0.64$, незначимо), что и есть нужный этой лестнице контроль. Четыре накопителя, выписанные в исходнике при неизменных флагах, в 1.13 раза *быстрее*, чем выдача компилятору общего разрешения переассоциировать ($t = -72.75$, значимо), восемь — в 1.18 раза медленнее четырёх и на 4.5 % медленнее сборки с `fast-math` ($t = 50.60$, значимо). То есть лестница и объясняет флаг, и обгоняет его, и у неё есть оптимум — здесь четыре, — которого общий флаг выразить не может. Цена в обоих случаях одинакова, и это не флаг: это другой порядок суммирования, и проверка корректности так его и записывает (ступени лестницы отклоняются от референса на $2-8 \cdot 10^{-15}$ относительно, сборка с `fast-math` — на $3.1 \cdot 10^{-15}$).

Почему этот набор измеряется дважды. Порядок, в котором `pyperf` проходит набор (§3), разносит первый зарегистрированный бенчмарк и последний по времени, и весь дрейф машины начисляется на отношение между ними — причём базовой линией каждого отношения на этом рисунке служит один из первых зарегистрированных бенчмарков. Поэтому набор прогоняется дважды, второй раз — с обращённым планом, оба прохода дописываются в один файл, так что каждый бенчмарк измеряется на обоих концах прогона. В этом прогоне получившийся разброс — относительный MAD от 0.05 до 0.32 % по набору, а побайтово одинаковые варианты сходятся в пределах 0.11 % друг от друга, именно это отделяет эффект архитектурного флага в 1.80 раза от шума и не оставляет вовсе ничего, что можно было бы сообщить про `-O2` против `-O3`.

Вывод. Три вещи, в порядке того, насколько они меняют решение. Во-первых, уровень оптимизации, который в строке сборки меняют первым, для ядер такой формы переменной не является: `-O2` и `-O3` — одна и та же программа на двух из трёх, а там, где не одна и та же, они и не разное время, — так что исследование, сообщающее разницу во времени между ними, сообщает собственный шум. Во-вторых, архитектурный флаг на этой цели — настоящая переменная, и он не является бесплатным выигрышем: на одном ядре он стоит $1.25\times$, на другом — ничего, а на третьем обходится в $1.80\times$. «Собирайте с `-march=native`» — поэтому не совет, а эксперимент, и ставить его надо для каждого ядра, дизассемблирование как раз и говорит, есть ли вообще на чём его ставить. В-третьих, единственный рычаг ценой в несколько раз — контракт арифметики с плавающей

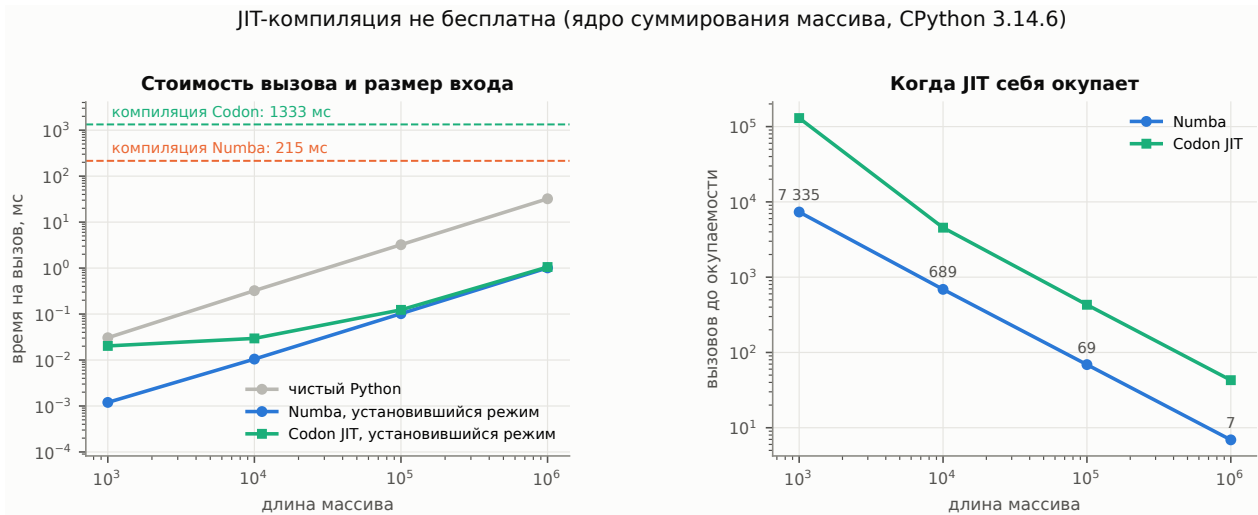


Рис. 3: В1в: установившаяся цена вызова в зависимости от размера входа для ядра на чистом Python и двух JIT, с разовой компиляцией каждого в виде горизонтальной линии, и получающиеся числа вызовов до окупаемости. Компиляция здесь — это декорирование функции и компиляция одной сигнатуры, импорт ускорителя относится к запуску и приводится отдельно.

точкой, и принадлежит он программе, а не компилятору: множитель больший, чем у флага ($2.74\times$ против $2.42\times$), доступен при `-O2` перегруппировкой накопления в исходнике, что выдаёт разрешение ровно там, где оно нужно, а не везде. Numba и clang — два фронтенда одного и того же LLVM, и измеряются они соответственно: на суммировании и на произведении матриц они согласуются с точностью около 1% и с `fast-math`, и без него (суммирование `-O2` против Numba $0.9981\times$, $t = 4.13$, `-ffast-math` против Numba `fastmath` $0.9978\times$, $t = 4.59$, произведение матриц `-O2` против Numba $1.0091\times$, $t = -12.59$ — все значимы при таком разрешении). На мандельброте Numba опережает `-O2` в 1.12 раза, а `-O3 -march=native -ffast-math` — в 1.10 раза с `fast-math`; но сборке `-O3 -march=native`, собранной под тот же процессор, она уступает в 1.11 раза, Numba по умолчанию компилирует под хостовый процессор, поэтому сравнение с `-O2` не равно с равным; остаток же против сборки под ту же цель — это различие кодогенерации между двумя фронтендами, а не свойство компиляции на лету. Одно ограничение идёт со всем этим вместе: `-ffast-math` изменил число итераций мандельброта на $4.6 \cdot 10^{-5}$ относительно — ровно та сделка, которая здесь и заключается.

4.2 В1в: чего стоит JIT, прежде чем окупится?

Постановка. Точка окупаемости без указания размера входа бессмысленна, поэтому мы измеряем одно и то же ядро суммирования при 10^3 , 10^4 , 10^5 и 10^6 элементах в одном процессе, отдельно измеряем разовую компиляцию у каждого JIT и вычисляем число вызовов окупаемости $k^* = t_{\text{компиляции}} / (t_{\text{Python}} - t_{\text{JIT}})$ для каждого размера. Измеряются два JIT — Numba и Codon. JIT из CinderX здесь намеренно отсутствует: его базой служит другая сборка интерпретатора, так что числитель и знаменатель брались бы из разных рантаймов, да и включается он не по решению программы для отдельной функции, а по порогу числа вызовов, независимо от чьего-либо желания, — отчего вопрос «сколько вызовов до окупаемости» теряет адресата, способного на него ответить действием.

Результат: точка окупаемости смещается с размером входа на три порядка. Рисунок 3. Компиляция сигнатуры в Numba стоит 215 мс. Времена вызова составляют $30.5 \mu\text{s}$ против $1.2 \mu\text{s}$ при 10^3 элементах и растут до 32.2 мс против 1.01 мс при 10^6 — устойчивые $26\text{--}32\times$. Точка окупаемости при этом смещается на три порядка: **7335** вызовов при 10^3 элементах, 689 при 10^4 , 69 при 10^5 и 7 при 10^6 . Для сервиса, который суммирует массивы из тысячи элементов несколько сотен раз на запрос, Numba — чистый убыток, если компиляция не амортизируется между запросами (или не кэшируется на диск), для случая с миллионом элементов она окупается почти сразу.

JIT из Codon выходит на тот же установившийся режим и берёт за вход примерно вшестеро больше: 1.33 с против 215 мс у Numba, из которых 0.55 с уходит на декорирование функции и 0.79 с — на первый вызов с новой сигнатурой. Оба числа отсчитываются от уже импортированного ускорителя до скомпилированной функции, сам импорт относится к запуску, берётся раз на процесс, а не на ядро, и приводится ниже. На диске у Codon не кэшируется ничего, так что следующий процесс повторяет всё заново и аналога `cache=True` у моста нет, а декорирование к тому же берётся с каждой функции, а не один раз. Числа окупаемости следуют отсюда: $1.3 \cdot 10^5$ вызовов при 10^3 элементах, 4545 при 10^4 , 430 при 10^5 и **43** при 10^6 — от шести до восемнадцати раз больше,

Таблица 4: B2: ветвистые ядра, ядра с выделением памяти и с нерегулярным доступом на CPython 3.14.6.

реализация	токенизация ($2 \cdot 10^6$ Б)		бинарные деревья ($d=18$)		BFS ($5 \cdot 10^5$ узлов)	
	мс	×	мс	×	мс	×
цикл на CPython	300.44	1.0	126.81	1.0	203.95	1.0
Cython	24.25	12.4	25.86	4.9	20.54	9.9
Codon AOT	25.75	11.7	7.37	17.2	21.90	9.3
Numba	26.11	11.5	32.36	3.9	23.17	8.8
Codon JIT	26.17	11.5	7.32	17.3	23.03	8.9
C (ctypes)	22.29	13.5	7.05	18.0	20.62	9.9
Rust (ctypes)	22.42	13.4	6.09	20.8	22.38	9.1

чем у Numba, в зависимости от размера, причём дело в задержке, а не в порождаемом коде.

Установившаяся цена у них тоже совпадает не везде, и здесь разрыв идёт в обратную сторону: при 10^6 элементах Codon и Numba расходятся на 5 % (1.054 против 1.010 мс), а при 10^3 Codon медленнее в 16.9 раза (20.3 против 1.2 μ с). Это граница вызова у JIT, а не его код: то самое пересечение масштаба микросекунд, за которое заранее собранное расширение не платит и которое незаметно на тех размерах, вокруг которых обычно и спорят о JIT.

Задержку компиляции не следует читать как константу ни в том, ни в другом случае. 215 мс у Numba — это то, чего стоит первая компиляция `njit` этой сигнатуры в процессе, который больше ничего не компилирует, и в неё входит ленивая инициализация самой Numba, за которую вторая компиляция уже не платит. У Codon разложение иное: декорирование — на функцию, первый вызов — на сигнатуру, так что «задержка компиляции» зависит от того, которое из двух программа собирается повторять.

Запуск — отдельная статья расходов. Импорт ускорителя не входит ни в одну окупаемость выше, но он не бесплатен, и различаются эти двое здесь сильнее всего: `import numba` стоит 190 мс, а `import codon` — 9.06 с. Большая часть второго — это компиляция Codon’ом собственной нативной реализации NumPy, которую инициализация моста импортирует безусловно, нужна она программе или нет, и штатного способа её пропустить не предусмотрено. Для долгоживущего сервиса это задержка запуска, оплаченная однажды и забытая, для утилиты командной строки, прогона тестов и всего прочего, что стартует новый процесс на каждую единицу работы, это девять секунд на единицу работы, и они перевешивают всё остальное, измеренное в этом разделе. Путь предварительной сборки не платит из этого ничего: модуль расширения, собранный `codon build -pyext`, импортируется за 73 мс, как любое другое расширение.

Вывод. Установившееся отношение — только половина решения. Уместность JIT зависит от произведения числа вызовов на работу в вызове, и на этом хосте порог охватывает три порядка для одного ядра и одного ускорителя — около 7 вызовов при 10^6 элементах и около 7000 при 10^3 — и ещё порядок между ускорителями: от 7 у Numba до 43 у Codon на том же ядре и том же размере. Любая рекомендация пользоваться JIT нуждается в этой второй координате — и, прежде чем любой из ускорителей начнёт что-либо возвращать, в цене самой его загрузки: 190 мс у одного из этих двух и девять секунд у другого.

5 B2: действительно ли ветвистым нагрузкам и нагрузкам с аллокациями нужен C?

Вопрос. Нерегулярный код — много условных переходов или структуры, до которых добираются по указателям, — это то место, где компиляция возвращает меньше всего. Какое свойство за это отвечает: ветвления, выделение памяти или характер доступа? И сколько даёт перенос такого кода за FFI, если считать и цену самой границы?

Постановка. Три ядра изолируют три разные трудности: *токенизация* насыщена ветвлениями, но не выделяет память (классификатор байтов с непредсказуемыми переходами по $2 \cdot 10^6$ байт), *бинарные деревья* насыщены выделением памяти (построить и свернуть дерево глубины 18: $2.6 \cdot 10^5$ мелких объектов, рекурсия, блуждание по указателям), *BFS* насыщен нерегулярным доступом (обход в ширину графа на $5 \cdot 10^5$ узлов с арифметически порождаемыми рёбрами). Каждая технология получает свой лучший шанс: Numba даётся с переписыванием под типы NumPy, а для дерева — с типом узла на `jitclass`.

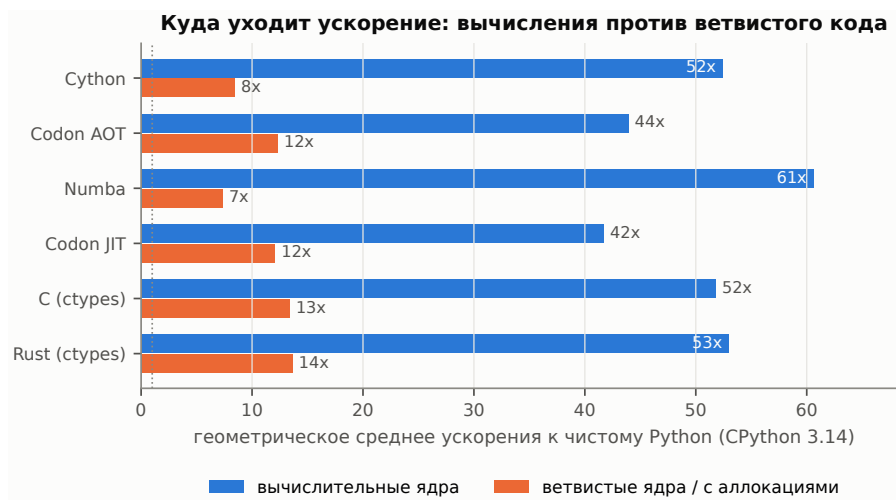


Рис. 4: B2: геометрическое среднее ускорения каждой технологии относительно чистого Python на трёх вычислительных ядрах против трёх ветвистых / с аллокациями.

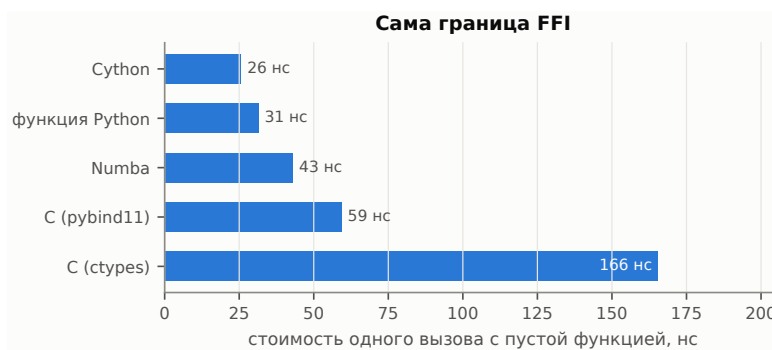


Рис. 5: B2: стоимость однократного пересечения каждой границы при пустой вызываемой функции.

Результат: три нерегулярных ядра ведут себя по-разному. Таблица 4 и рисунок 4. В среднем нерегулярные нагрузки действительно хуже поддаются компиляции: геометрическое среднее ускорения относительно чистого Python падает с 42–61× на вычислительных ядрах до 7–14× здесь. Но среднее скрывает самое интересное: три ядра ведут себя по-разному, и различающее их свойство — не то, на которое указывает выражение «ветвистый код».

С непредсказуемым ветвлением по типизированному массиву компилятор справляется прекрасно. На *то-кенизации* — самом ветвистом ядре, с автоматом состояний, зависящим от данных, по 2 МБ псевдослучайных байт — все шесть компилируемых реализаций укладываются в 18 % друг от друга: C 13.5×, Rust 13.4×, Cython 12.4×, Codon 11.7× при предварительной сборке и 11.5× под JIT, Numba 11.5×. Они близки, но не равны, и попарные проверки разделяют все пятнадцать пар (Numba против C 0.8537× при $t = 123.9$, Cython против C 0.9192× при $t = 98.5$, Codon против C 0.8656× при $t = 164.9$, C против Rust 1.0060× при $t = -9.53$). Дело в размере разрыва, а не в его существовании: не более 17.4 % против тех 13×, которых стоит компиляция цикла. Компилятор прекрасно справляется с непредсказуемыми переходами по типизированному массиву.

Препятствие — выделение памяти, а решает дело то, кто этой памятью владеет. На *бинарных деревьях*, где на вызов создаётся $2.6 \cdot 10^5$ мелких объектов, шесть компилируемых реализаций делятся надвое без промежуточных случаев: с одной стороны Cython с 4.9× и Numba с 3.9×, с другой — Codon с 17.2× при предварительной сборке и 17.3× под JIT, C с 18.0× и Rust с 20.8×. Через эту границу — разница в 3.5–5.3 раза, и попарные проверки разделяют её во всех направлениях (Codon быстрее Cython в 3.51 раза и Numba в 4.39 раза, обе разницы при $|t| > 240$, Codon отстаёт от C на 4.4 %, $0.9563 \times$, $t = 9.64$).

Граница проходит по владению памятью, а не по противопоставлению «нативное против Python». Узел в Cython — это `cdef class`, в Numba — `jitclass`, оба типизированы, и оба остаются объектами со счётчиком ссылок, которые выдаёт аллокатор CPython. Узел в Codon — обычный класс с двумя полями `optional`: исходник на Python, без `jitclass`, без распрямления в массивы, — и он оказывается рядом с C и Rust, потому что класс Codon есть нативный объект в куче, за которым не стоит никакого счётчика ссылок. Значение имеет не типизация узлов, а замена аллокатора, — и для неё уходить от исходника в форме Python не требуется.

Codon служит здесь и контролем для конкурирующего объяснения. Если бы группы делил момент компи-

Таблица 5: B2: токенизатор, дизассемблированный из обеих разделяемых библиотек. Подсчитываемые мнемоники специфичны для архитектуры и записаны вместе с ней в `results/codegen/summary.json`.

ядро токенизации	инструкций	условных переходов	условных пересылок	мс
C, clang -O3 -march=native	167	20	6	22.29
Rust, rustc -C opt-level=3 -C target-cpu=native	180	20	6	22.42

Дизассемблировано на x86_64; подсчитываемые мнемоники переходов и пересылок записаны в `results/codegen/summary.json`.

ляции, а не то, что код размещает в памяти, две его строки разошлись бы, они не расходятся — 7.37 против 7.32 мс, различия критерий значимости не разрешает.

Нерегулярный доступ к памяти тоже не препятствие: на BFS по графу из $5 \cdot 10^5$ узлов Cython ($9.93\times$) и C ($9.89\times$) расходятся меньше чем на полпроцента — различие значимо, но ничтожно ($1.0042\times$, $t = -4.02$), — а Codon ($9.31\times$ и $8.86\times$), Rust ($9.11\times$) и Numba ($8.80\times$) идут недалеко позади.

Напрашивающееся объяснение — что все шесть отказались от двусторонней очереди в пользу заранее выделенного массива, так что измеряется размещение данных, а не язык. Мы это объяснение измерили, а не приняли на веру, потому что каждая компилируемая реализация здесь меняет обе вещи сразу. `bfs_py_flat` — тот же обход на чистом Python с очередью в виде заранее выделенного списка — тот же алгоритм, тот же порядок посещения, тот же результат, — и он занимает 228.5 мс против 204.0 мс у версии с очередью: плоское размещение в 1.12 раза медленнее ($t = 60.74$, значимо). Распрямление структуры данных само по себе и на чистом Python на этом ядре является регрессией, все $9.9\times$ — это стоимость интерпретации цикла. Правило «сначала распрямите структуры данных» здесь не подтверждается, подтверждается другое — плоское размещение это то, что *позволяет* цикл скомпилировать, а множитель берётся из компиляции.

Три меньших различия стоит записать, потому что они пересекаются с B1b, а не с языками. C и Rust, собранные из одного алгоритма на эквивалентных уровнях оптимизации, расходятся меньше чем на 1 % на токенизации (Rust на 0.6 % медленнее, значимо), в 1.16 раза в пользу Rust на бинарных деревьях и в 0.92 раза против него на BFS. Различия такого размера между двумя тулчейнами на основе LLVM — это кодогенерация, а не свойство любого из языков, и они меньше, чем эффекты флагов, которые §4.1 измеряет внутри одного компилятора.

На токенизатор стоит взглянуть ещё раз, потому что это ядро, на котором эти двое могли бы правдоподобно разойтись: это автомат состояний по непредсказуемому входу, поэтому интересен вопрос, порождает ли компилятор цепочку условных переходов, которая ошибается в предсказании, или бесцветные условные пересылки, которые не ошибаются. Таблица 5 дизассемблирует оба. Форма ветвления у них одна и та же — по двадцать условных переходов и по шесть условных пересылок, — при том что всего инструкций 167 против 180. Значит, те 0.6 %, на которые они всё-таки расходятся, предсказанием переходов не объясняются: различие реально, но лежит не там, где его естественнее всего было бы искать.

Сама граница. Рисунок 5 измеряет один вызов с пустой вызываемой функцией: Cython 26 нс, обычная функция Python 31 нс, диспетчер Numba 43 нс, функция `rybind11` 59 нс и `stypes` 166 нс. Два следствия. Во-первых, FFI-вызов не бесплатен, но дешёв: при $\approx 10^2$ нс ядро, выполняющее ≥ 20 μ с работы, платит за пересечение меньше 1% даже через `stypes`, а для более дешёвых привязок хватает и ≥ 6 μ с. Во-вторых, `stypes` — самый удобный способ вызвать нативный код — оказывается самой дорогой границей, в 5.3 раза дороже вызова уровня Python, поэтому FFI-вызовы на каждую запись — реальная статья расходов: 20 000 вызовов `stypes` по записям в §10 тратят ≈ 3.3 мс только на пересечения границы.

Вывод. Класс надо разделить на два, потому что различающее свойство — *кто владеет памятью*, а не сколько в коде ветвлений. Ветвистому проходу по плоским данным C не нужен: Cython, Codon и Numba дотягиваются до него в пределах 18 %. На графе объектов с интенсивным выделением памяти C и Rust уходят вперёд от Cython и Numba в 3.7–5.3 раза, и закрывает этот разрыв смена модели объектов, а не её типизация. Codon показывает, что для этого не требуется уходить от исходника в форме Python: $17.2\times$ против $18.0\times$ у C, из класса с двумя полями `optional`. Поэтому рекомендация выносить код с интенсивными аллокациями за FFI поддержана лишь в слабой форме: такому коду нужен аллокатор, отличный от CPython'овского, а это не то же самое, что нужен C.

6 В3: что дало объектному коду развитие CPython 3.10–3.14?

Вопрос. Специализация байткода, встраивание вызовов, бессмертные объекты и работа над аллокатором появились в CPython между 3.10 и 3.14. Что они дали объектно-нагруженному коду, какой выпуск это дал и распределено ли это равномерно по операциям или сосредоточено в нескольких?

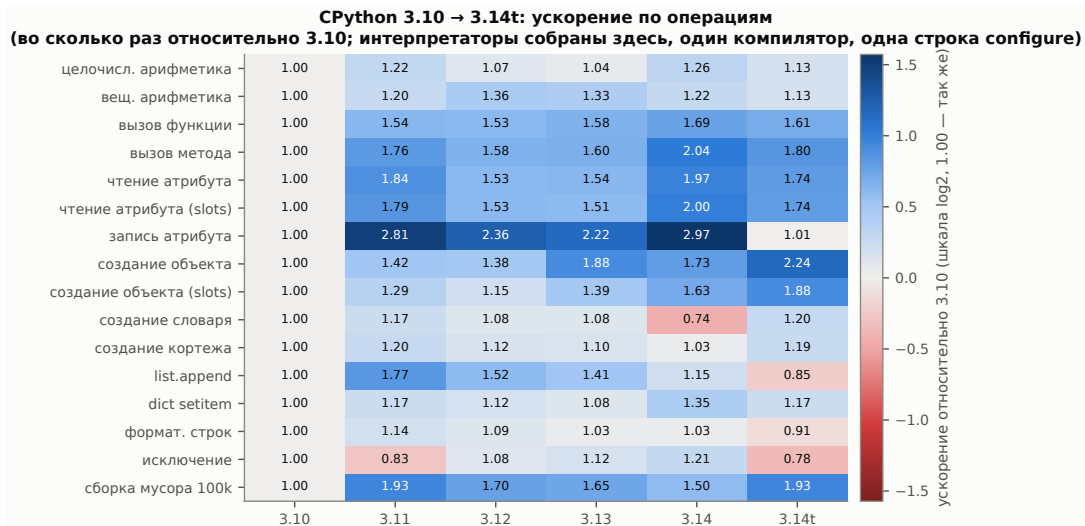


Рис. 6: ВЗ: ускорение по операциям относительно CPython 3.10 (значения — во сколько раз, синий — быстрее, красный — медленнее).

Таблица 6: ВЗ: структурные свойства рантайма, измеренные тем же набором.

свойство	3.10	3.11	3.12	3.13	3.14	3.14t
sys.getrefcount (None)	7 780	7 887	4 294 967 295	4 294 967 295	3 221 225 472	3 221 225 472
None бессмертен	нет	нет	да	да	да	да
блоков аллокатора / объект (dict)	5.96	4.96	4.96	3.96	3.96	3.96
блоков аллокатора / объект (slots)	3.96	3.96	3.96	3.96	3.96	3.96
макс. RSS после набора МБ	27	28	30	28	30	36
чистый запуск мс	9.0	9.4	10.5	10.6	10.6	13.7

Постановка. Набор только на стандартной библиотеке (чтобы единственной переменной был интерпретатор), выполненный на CPython 3.10.18, 3.11.6, 3.12.12, 3.13.5, 3.14.6 и free-threaded 3.14.6t: создание экземпляров в представлениях со словарём, с `__slots__`, обычного словаря и кортежа, чтение и запись атрибута, вызовы связанного метода и обычной функции, циклы целочисленной и вещественной арифметики, операции со списком, словарём и строками, проход исключения, пауза сборки мусора по 10^5 объектам в ссылочных циклах, плюс неизмерительные структурные пробы (счётчики ссылок синглтонов, блоки аллокатора на экземпляр, задержка запуска).

Почему интерпретаторы пришлось собирать здесь. Сравнение версий является сравнением версий только тогда, когда вместе с версией не меняется ничего другого, а опции сборки интерпретатора стоят не меньше, чем несколько выпусков работы над интерпретатором (§9). Поэтому каждая версия этого раздела собрана из своего релизного архива скриптом `bench/build_uniform.sh` одним компилятором и одной строкой `configure --enable-optimizations`, записанной одинаково в каждом выпуске от 3.10 до 3.14, — так что единственное различие между шестью интерпретаторами — это исходники, из которых они собраны. Free-threaded сборка отличается ровно ещё одним токеном, `--disable-gil`, и приводится отдельно в §8.

Результат: выигрыш реален и пришёл не выпуск за выпуском. Рисунок 6, относительно 3.10. Геометрическое среднее по шестнадцати операциям составляет $1.44\times$ на 3.11, $1.35\times$ на 3.12, $1.38\times$ на 3.13 и $1.45\times$ на 3.14:

- **3.11 — это ступенька**, как и предсказывает работа над специализирующим интерпретатором [5]: запись атрибута $2.81\times$, чтение атрибута $1.84\times$, `list.append` $1.77\times$, вызовы связанного метода $1.76\times$, вызовы функций $1.54\times$, сборка циклического мусора $1.93\times$. В совокупности она уже достигает $1.44\times$ против тех $1.45\times$, которые три следующих выпуска в итоге дают.
- **3.12 частично отдала это назад**, и отдача не маргинальна: целочисленная арифметика падает с $1.22\times$ до $1.07\times$, чтение атрибута с $1.84\times$ до $1.53\times$, запись атрибута с $2.81\times$ до $2.36\times$, `list.append` с $1.77\times$ до $1.52\times$. На 3.12 улучшаются две операции: вещественная арифметика, достигающая там максимума $1.36\times$ и дальше держащаяся около него, и круговой обход исключений, выигрывающий у 3.11 $1.30\times$.

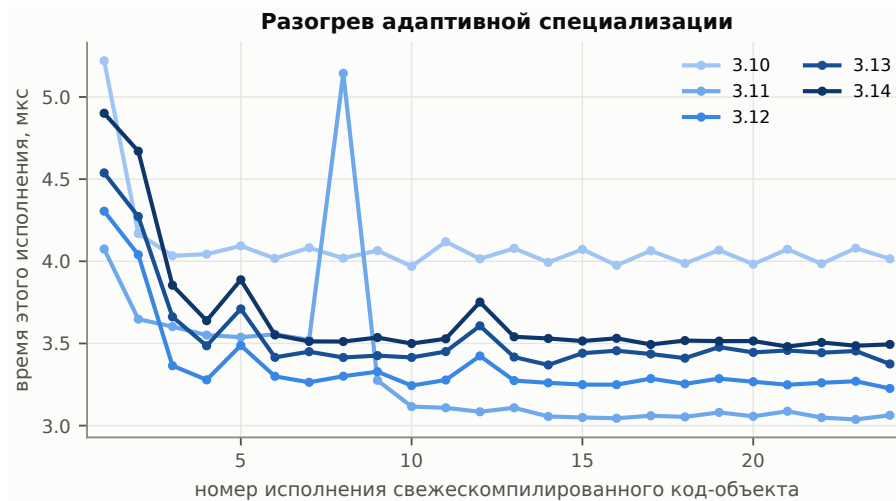


Рис. 7: ВЗ: время одного исполнения свежекомпилированного код-объекта на первых его исполнениях, по выпускам. Каждая точка — отдельный бенчмарк `rupegf`, измеряемая область которого и есть это одно исполнение.

- **3.14 — лучший выпуск серии для большинства объектных операций.** Она ставит максимум на вызовах методов ($2.04\times$), чтении атрибута ($1.97\times$ со словарём, $2.00\times$ со слотами), записи атрибута ($2.97\times$), вызовах функций ($1.69\times$), целочисленной арифметике ($1.26\times$), присваивании элемента dict ($1.35\times$) и проходе исключения ($1.21\times$). Против 3.13 это крупные движения за один выпуск — чтение атрибута $1.29\times$, вызовы методов $1.28\times$, запись атрибута $1.34\times$, все значимы, — так что форма зависимости по выпускам — ступенька на 3.11, провал через 3.12–3.13 и вторая ступенька на 3.14, а не полка.
- **Восстановлено не всё, и одна операция заканчивает ниже, чем начала.** Построение dict-литерала даёт $0.74\times$ от 3.10, то есть на 35 % медленнее (с 4.48 до 6.05 мс на 20 000 словарей), и вся эта регрессия приходит на 3.14 (в 1.46 раза медленнее 3.13, $t = -128$). `list.append` достиг максимума на 3.11 ($1.77\times$) и составляет $1.15\times$ на 3.14, сборка циклического мусора достигла максимума на 3.11 ($1.93\times$) и составляет $1.50\times$.
- **Free-threaded сборка — это другой профиль, а не равномерно более медленный.** Её геометрическое среднее — $1.33\times$, но это лучшая измеренная конфигурация для создания экземпляров (обычные объекты $2.24\times$, со слотами $1.88\times$), и она совпадает с 3.11 на сборке мусора ($1.93\times$), тогда как запись атрибута сваливается до $1.01\times$, то есть до цифры 3.10, а `list.append` ($0.85\times$), проход исключения ($0.78\times$) и форматирование строк ($0.91\times$) заканчивают ниже 3.10.

Какие из механизмов видны в самом рантайме. Таблица 6 содержит структурные пробы. Бессмертные объекты (PER 683 [12]) отчётливо присутствуют с 3.12 — `sys.getrefcount(None)` прыгает с $8 \cdot 10^3$ до $2^{32}-1$, а на 3.14 — до 3 221 225 472, — и всё же 3.12 медленнее 3.11 почти на каждой измеренной операции. Бессмертность убирает записи счётчика ссылок для разделяемых синглтонов, что окупается при многопоточной конкуренции, а не в однопоточных циклах, наши измерения просто не задействуют то, что она улучшает.

Работа над аллокатором, напротив, видна и объясняет тихо истёкшее эмпирическое правило. Блоков аллокатора на экземпляр становится 5.96 (3.10), затем 4.96 (3.11–3.12) и 3.96 (3.13–3.14) для экземпляра со словарём, тогда как экземпляр с `__slots__` остаётся на 3.96 всё время: к 3.13 обычный объект стоит аллокатору ровно столько же, сколько объект со слотами. Во времени создания преимущество `__slots__` сокращается с 25% (3.10) до 14% (3.11) и 4% (3.12) и на 3.13 даже *обращается*: там объекты со слотами на 8% медленнее объектов со словарём (6.48 против 5.98 мс на 20 000), после чего на 3.14 возвращается преимущество в 18%, — при том что структурная проба говорит, что аллокатору обе формы по-прежнему стоят одинаково. Значит, механизм, которым это правило обосновывают — на одну аллокацию меньше на экземпляр, — перестал действовать на 3.13. Разница во времени создания при этом никуда не делась: на 3.14 `__slots__` по-прежнему стоят 18%, но по причине, которую эти измерения не устанавливают. Предпоследняя строка таблицы 6 — та, которая не приносит, а стоит: пиковый RSS после того же набора растёт с 27 МБ на 3.10 до 30 МБ на 3.14 и до 36 МБ на free-threaded сборке — на пятую часть больше памяти у той сборки, которую измеряет §8, и берётся она независимо от того, запустится ли второй поток.

Рисунок 7 строился, чтобы сделать адаптивный интерпретатор непосредственно видимым, и он этого не делает. На свежекомпилированной линейной арифметической функции первое исполнение стоит $1.30\times$ установившегося уровня на 3.10, $1.33\times$ на 3.11, $1.32\times$ на 3.12 и 3.13 и $1.40\times$ на 3.14, — а на 3.10 разогревать специализацию нечему. Расход на первое исполнение такого размера присутствует, значит, и с адаптивным интерпретатором, и без него, так что эта проба не может отнести его к специализации, она измеряет стоимость

первого исполнения код-объекта, что бы интерпретатор с ним ни делал. Тот же рисунок несёт более чистый результат: сам установившийся уровень резко улучшился на 3.11 — с $4.03\ \mu\text{s}$ на 3.10 до $3.06\ \mu\text{s}$, — и с тех пор дрейфовал обратно до $3.51\ \mu\text{s}$ на 3.14, так что линейный целочисленный код на 3.14 в 1.15 раза быстрее, чем на 3.10, будучи разогретым, и в 1.15 раза *медленнее*, чем на 3.11.

Запуск двигался по серии в другую сторону, и читать его надо рядом с пропускной способностью: чистый старт интерпретатора стоит 9.0 мс на 3.10 и 10.6 мс на 3.14, импорт стандартного набора модулей — 47.6 против 66.3 мс, а free-threaded сборка снова медленнее (13.7 и 75.8 мс). Для долгоживущего сервиса это невидимо, для утилиты командной строки, вызываемой на каждый файл, это единственное число этого раздела, которое имеет значение.

Вывод. Обновление интерпретатора для объектного кода действительно окупается: операции с атрибутами и методами на 3.14 быстрее, чем на 3.10, в 1.97–2.97 раза, а геометрическое среднее по всему набору — $1.45\times$. К этому идут уточнения. Выигрыш распределён по выпускам неравномерно: большую его часть даёт 3.11, 3.12 и 3.13 часть отдадут назад, а 3.14 возвращает её и добавляет ещё. Поэтому «обновляйтесь» — хороший совет, а «каждый выпуск помогает» — нет. Из обычно называемых механизмов наши измерения относят выигрыш к специализирующему интерпретатору и к аллокатору, бессмертные объекты доказуемо присутствуют с 3.12 и доказуемо невидимы для однопоточного кода. Там, где они должны окупаться — в конкуренции за счётчик ссылок между потоками, — работа их не выделяет, поэтому здесь можно сказать лишь то, что эти числа сдвинули не они. И к этому выводу нужно приложить два предупреждения: одна операция нашего набора на 35 % медленнее на 3.14, чем на 3.10, и стала такой в последнем выпуске, а запуск с импортом стандартной библиотеки за тот же свип стал на 39 % медленнее, — так что обновление заслуживает собственного измерения на том коде, который важен.

7 B4: Cinder и CinderX — какие возможности окупаются и какой ценой?

Вопрос. Форк Cinder от Meta и его расширения CinderX предлагают ленивые импорты, облегчённые фреймы, параллельный сборщик мусора, JIT и режим статической типизации с распакованными примитивами. Что из этого окупается на обычном коде, что требует переписывания кода и чего стоит тому, кто решит его внедрять, текущее состояние открытой версии проекта?

Постановка. Обычный CPython 3.14.6 и форк Cinder от Meta (meta/3.14, 3.14.5+meta) собраны из вложенных в артефакт исходников на этом хосте с одинаковыми флагами `configure` и одним компилятором (clang 18.1.3), так что разница — только набор патчей Meta, ни один из них не получает PGO. Затем против форка собирается пакет расширений CinderX — дважды, потому что `ENABLE_ADAPTIVE_STATIC_PYTHON` открывает специализирующий путь собственного интерпретатора CinderX и потому прямо относится к производительности. Каждая конфигурация ниже измерена и с выключенным этим флагом, и с включённым, каждая — в собственном дереве, но обе из одного исходника и за один проход. `ENABLE_EVAL_HOOK` выключен в обеих: это механизм до 3.13, который на 3.14 заменён PEP 523 [13], — выключенное состояние здесь правильное, а не ограничение. Конфигурации измеряются на тех же шести ядрах, что B1/B2, плюс собственные функции форка (ленивые импорты, параллельный GC, работа с фреймами) и Static Python.

То, как именно JIT просят скомпилировать функцию, относится к постановке, а не к мелочам, потому что от этого зависит ответ. JIT компилирует тот байткод, который ему показали, а CPython 3.14 переписывает байткод по ходу исполнения функции, — значит, функция, скомпилированная до первого вызова, скомпилирована из полностью обобщённого байткода. Поэтому все строки с JIT ниже используют собственный порог JIT (`compile_after_n_calls`), после чего в том же процессе, который будет мерить, проверяются две вещи: что ядро скомпилировано и что ещё один вызов не увеличивает `count_interpreted_calls`. Несущая здесь вторая — об этом в конце раздела.

Результат: сам форк — не главное. Собранный теми же флагами, форк Cinder укладывается в 2.1 % от обычного CPython 3.14.6 на всех шести ядрах ($0.98\text{--}1.01\times$ его времени, рисунок 8 сверху). Пять из этих шести различий критерий значимости разрешает; шестое (двоичные деревья, $0.9996\times$ при $t = 0.15$) — нет. Честная формулировка поэтому не «то же самое», а «измеримо различно и без пользы»: крупнейшее из различий — сумма массива на $0.979\times$ ($t = 5.2$), то есть форк там на 2.1 % быстрее, следом произведение матриц с $0.982\times$, и ни в одну сторону ничего не сдвигается больше чем на 2.1 %. Глубокие функции его рантайма включаются по требованию, и здесь окупается одна — ленивые импорты: с ними импорт пакета из 200 модулей исполняет 0 тел модулей вместо 200 и падает с 19.7 мс до $0.63\ \mu\text{s}$, а первое обращение к отложенному модулю стоит 0.35 мс. Это функция задержки запуска, и у обычного CPython аналога нет — наша проба на нём записана как `unavailable`

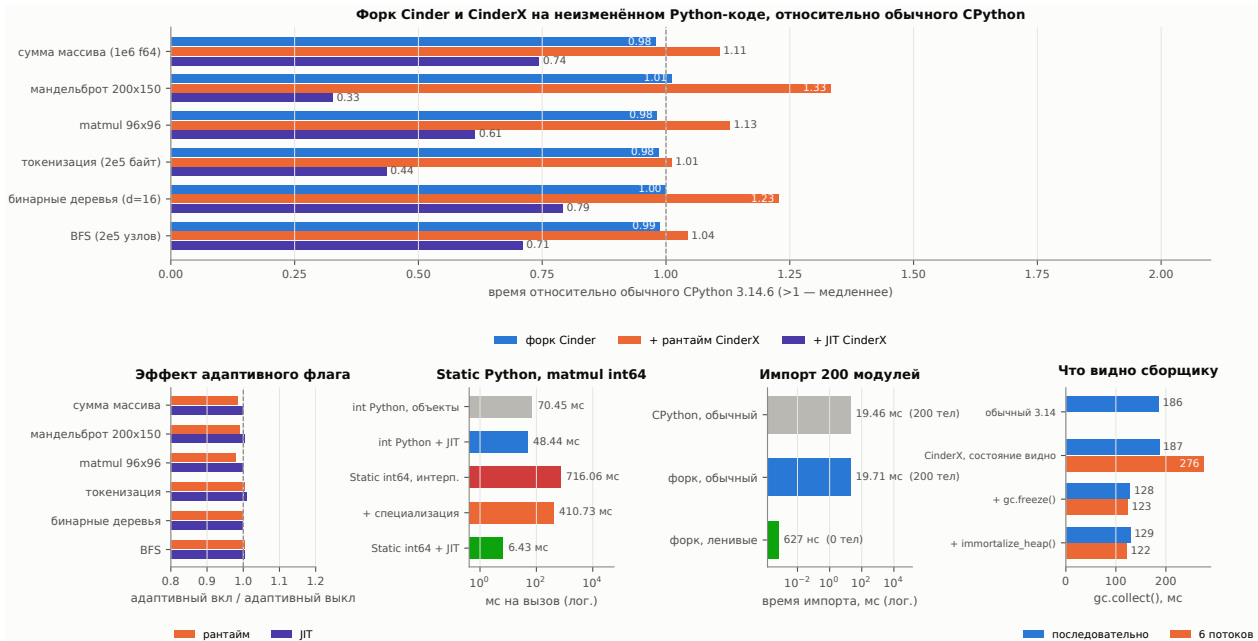


Рис. 8: В4: (сверху) форк, рантайм CinderX и JIT CinderX на неизменённом Python-коде относительно обычного CPython 3.14.6, собранного теми же флагами, (снизу слева) влияние флага адаптивного Static Python на те же отношения, (снизу в центре) целочисленный matmul на Static Python int64 (построено для медленной раскладки, 6.43 мс; быстрая — 4.25 мс, §7) — против того же ядра с объектами и того же ядра с объектами под JIT, а также в интерпретаторе без адаптивной специализации и с ней, (снизу правее центра) задержка импорта пакета из 200 модулей с ленивыми импортами форка и без них, (снизу справа) одна сборка кучи воркера — долгоживущее состояние плюс мусор на запрос — при видимом сборщику состоянии, после `gc.freeze()` и после `immortalize_heap()`, последовательно и на шести потоках, с обычным CPython на той же куче для сравнения.

(`importlib.set_lazy_imports()` там не существует), а не как более медленное время. Машинерия облегчённых фреймов здесь, наоборот, не даёт ничего полезного: цепочка вызовов глубиной 200 — $1.01\times$ обычной сборки, трассировка из 50 фреймов — $0.97\times$, обход 100 фреймов — $1.01\times$; все три расхождения тест разрешает, но полезной величины нет ни у одного. Параллельному сборщику отведён отдельный абзац ниже: одно число оказалось для него ответом не на тот вопрос.

Результат: рантайм обходится дорого, JIT эту цену с запасом возвращает, а адаптивный флаг не меняет ни того, ни другого. Инициализация рантайма CinderX — она ставит собственный вычислитель фреймов вместо того, который CPython специализирует, — делает те же ядра в 1.01 – 1.33 раза *медленнее* обычной сборки, хуже всего на сетке манделъброта. Под рантаймом пробы на фреймах стоят дороже, чем сами ядра: цепочка вызовов глубиной 200 — $1.37\times$, трассировка из 50 фреймов — $1.15\times$, обход 100 фреймов — $1.13\times$, это та же машинерия, которая на голом форке была нейтральной, так что цена приходит с расширением, а не с набором патчей.

JIT возвращает эту цену с запасом, и на каждом ядре: 0.33 – $0.79\times$ обычной сборки (рисунок 8 сверху), то есть быстрее в 1.26 – 3.06 раза. Порядок показателен, но не безупречен. Больше всех выигрывает сетка манделъброта ($0.33\times$), меньше всех — двоичные деревья, которые выделяют $6.6 \cdot 10^4$ мелких объектов на вызов ($0.79\times$): метод-JIT снимает издержки интерпретации, а выделение памяти к ним не относится. Но выделение памяти не единственное, что его ограничивает: сумма массива выигрывает почти столь же мало ($0.74\times$), ничего при этом не выделяя, а обход графа — заметно больше ($0.71\times$), выделяя много. Там, где предел ставит именно память, §5 измеряет, чего стоит его сдвинуть: Codon, компилирующий сопоставимый исходник, но размещающий свои узлы нативно, проходит двоичные деревья в 17.2 раза быстрее чистого Python там, где JIT CinderX даёт 1.26 раза, — на дереве глубины 18 против глубины 16 в В4, так что эти два множителя показательны, но несопоставимы напрямую. JIT, сохраняющий объекты CPython, работает здесь над меньшей половиной задачи, и никакая компиляция до второй половины не достаёт. Единственное место, где он помогает неравномерно, — пробы на фреймах: цепочка глубиной 200 даёт $0.85\times$, трассировка из 50 фреймов $0.98\times$, обход 100 фреймов $1.33\times$.

Адаптивный флаг здесь ни при чём ни в ту, ни в другую сторону. «Включён к выключенному» лежит в 0.98 – 1.00 под рантаймом и в 1.00 – 1.01 под JIT (рисунок 8, снизу слева), а относительно обычной сборки диапазоны составляют 1.01 – $1.32\times$ и 0.33 – $0.79\times$ против 1.01 – 1.33 и 0.33 – 0.79 без него. На нетипизированном коде обе конфигурации — один и тот же интерпретатор.

Таблица 7: В4: что на самом деле даёт открытый CinderX, собранный и запущенный нативно на хосте из таблицы 1.

возможность	итог	измерение
собирается на Linux x86_64	да	0 ошибок в обеих конфигурациях, с патчами, которые несёт приложенное дерево
ЛТ компилирует и даёт верный результат	да	все 8 функций ядер, скомпилированы по собственному порогу ЛТ, с проверкой, что исполняются именно они
рантайм CinderX без ЛТ	цена	matmul 1.13× времени обычной сборки, 1.01–1.33× по шести ядрам
ЛТ CinderX на нетипизированном коде	выигрыш	matmul 0.61× времени обычной сборки, 0.33–0.79× по шести ядрам
то же, адаптивная конфигурация	выигрыш	matmul 1.11× / 0.61× времени обычной сборки
Static Python + ЛТ	выигрыш	11.4× быстрее того же ядра с объектами под тем же ЛТ (7.5× на медленном режиме раскладки)
Static Python, интерпретатор	цена	10.2× медленнее Python с объектами
адаптивная специализация, после до-работки	выигрыш	статический интерпретатор 716 → 411 мс; под ЛТ эффекта нет
ленивые импорты	выигрыш	19.7 мс → 0.63 мкс; 0/200 тел модулей исполнено; первое обращение 0.35 мс
параллельный GC	зависит от состава кучи	0.68× от последовательного при видимом состоянии, 1.06× после её иммортализации (187 → 129 мс последовательно)
облегчённые фреймы	нет эффекта здесь	цепочка вызовов глубиной 200 1.01× обычной сборки

Результат: конфигурация по умолчанию и есть конфигурация. ЛТ принимает 37 задокументированных опций — через `-X cinderx-jit-...` или переменные окружения, — и вся документация к ним состоит из однострочной подсказки в исходнике. Мы измерили все 37 по одной и одиннадцать композиций, подобранных по тому, как опции могли бы разблокировать друг друга, а не перебором: увеличенный бюджет инлайнера вместе с ограничителями, которые его сдерживают, проверки по аннотациям вместе с этим бюджетом, разделение горячего и холодного кода вместе с тем отображением страниц, ради которого оно и существует. *Быстрее конфигурации по умолчанию нет ничего.* Девять значимых конфигураций затем перемерены по протоколу самой работы. На рисунке 9 показаны те, которые читатель мог бы реально изменить, конфигурация, просто выключающая оптимизацию, медленнее, и это не результат, поэтому такие измерены, но не вынесены на схему.

Крупнейший эффект — ровно тот, которого никто не ожидает: если попросить ЛТ компилировать *раньше*, становится медленнее. Компиляция каждой функции при первом вызове — опция называется `jit-all` — исполняет шесть ядер со скоростью 0.49× от конфигурации по умолчанию, а на сетке мандельброта — 0.23×. И потерю удаётся опознать: запрет компилировать специализированные опкоды даёт 0.49×, причём на каждом ядре эти два числа совпадают с точностью до 0.02. Значит, штраф, который мы поначалу сообщили как свойство ЛТ, — это ровно одна потерянная оптимизация, и любому пользователю достаточно одного флага `-x`, чтобы её вернуть. Ничто из того, что можно включить, тоже не помогает: разделение горячего и холодного кода даёт 0.96×, предназначенное для него отображение на большие страницы этого не меняет, а отключение `inline-кэшей` доступа к атрибутам — 0.97× в целом, но 0.87× на двоичных деревьях и 0.92× на обходе графа.

Одна группа опций во всём этом отсутствует, и причина в нас, а не в ЛТ. Четыре из 37 настраивают инлайнер: его бюджет по стоимости, сколько зависимых функций подгружается вместе с компилируемой, когда место вызова отбрасывается как холодное, как далеко разрешено уходить упрощателю. На наших шести ядрах инлайнеру почти нечего делать: если спросить его после компиляции, он сообщает две встроенные функции на весь набор, обе в двоичных деревьях, и ни одной в остальных пяти — их горячие циклы не зовут Python-функций. Подъём бюджета вдесятеро при открытых воротах отсева и поднятом пределе предзагрузки оставляет и эти числа, и размеры скомпилированных функций неизменными до единицы. Значит, четыре опции инертны здесь по построению, и это утверждение о нагрузке, а не о них: на единственном ядре, которое инлайнер задействует, его отключение даёт 0.91×, а дальнейший подъём бюджета не даёт ничего — так выглядит насыщенная оптимизация.

Результат: флаг вышел с половиной механизма, и вторую половину пришлось дописать нам. На нетипизированном коде флаг не меняет ничего, как показано выше. На коде, ради которого он существует, он делал хуже, чем ничего: на коммите, с которого мы начали, `ENABLE_ADAPTIVE_STATIC_PYTHON=1` означало, что Static Python не исполняется вовсе. Типизированный модуль импортировался и компилировался, а первое же исполнение примитивного накопления `s = s + a[i*n+k] * b[k*n+j]` выбрасывало `TypeError: 'int' object is not iterable`. Обычный Python с объектами на том же интерпретаторе работал нормально, то есть отказ был свой-

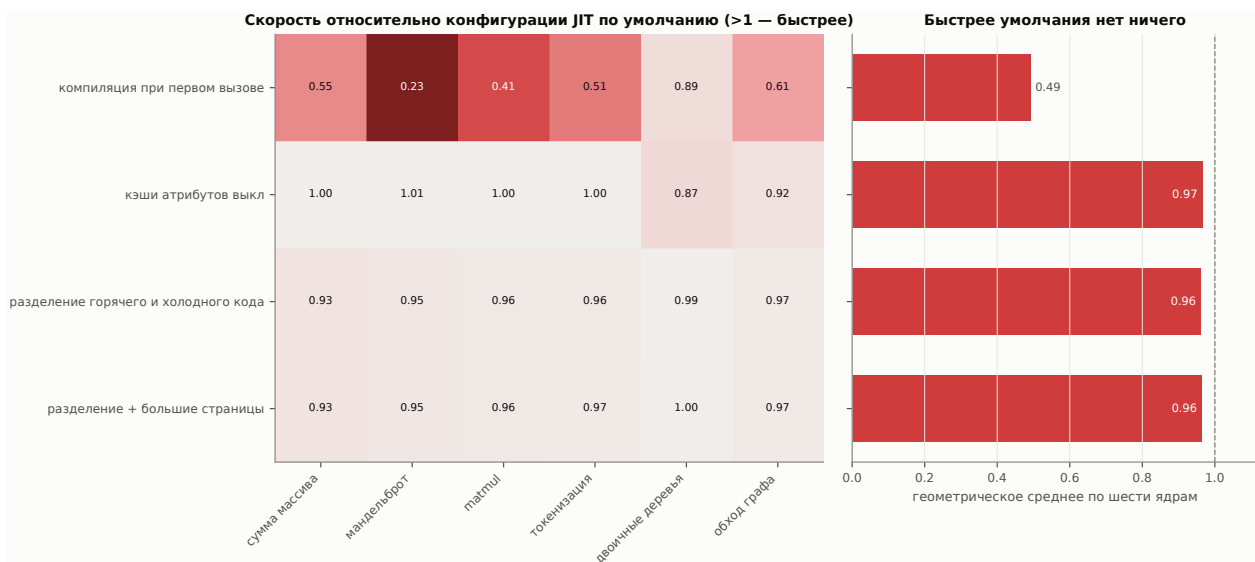


Рис. 9: B4: настройки JIT, которые читатель мог бы действительно изменить, относительно той конфигурации, в которой измерен весь остальной раздел. Слева по ядрам, справа геометрическое среднее по шести. Пять конфигураций, существующих лишь для того, чтобы определить, какую оптимизацию теряет другая настройка — `no-jit`, `no-inliner`, `no-specialized-opcodes`, `no-simplifier` и `no-stable-frame`, — измерены, но на схему не вынесены; две из них, которые обнаруживают потерю, приводятся в тексте.

ством именно того пути, который флаг и должен ускорять.

Доложить это как приговор проекту было бы дёшево и неверно, потому что «функция сломана» и «функция перенесена наполовину» — разные сведения для того, кто решает, брать ли её. Верно второе. Интерпретатор 3.14 нёс двенадцать мест, где статический опкод специализируется — семь `_Ci_specialize` и пять `specialize_with_value`, — и ни одного обработчика для опкодов, в которые они специализируются, интерпретатор 3.12 в том же дереве имеет восемь таких обработчиков, то есть механизм существовал, а перенесли его в одну сторону. Следствия отсюда, каждое смертельное само по себе. `_Ci_specialize` записывал кэшированный опкод поверх префикса `EXTENDED_OPCODE`, а не поверх самого статического опкода, — и младший байт статического опкода является обычным опкодом CPython: `STORE_LOCAL_CACHED` равен 16, а опкод 16 в обычной таблице — это `GET_ITER`, так что следующее исполнение той же инструкции уходило в `GET_ITER` на сохраняемом числе, что и даёт наблюдавшийся `TypeError`. Статический компилятор не резервировал ячеек inline-кэша для тех опкодов, чьи специализации кэш пишут, поэтому `CAST` записывал свои 32 бита поверх двух следующих инструкций. А диспетчер в любом случае перешагивал ровно одну кодовую единицу.

Чтобы это доделать, понадобились обработчики для одиннадцати достижимых специализированных опкодов, одна общая таблица длины статической инструкции — ею пользуются интерпретатор, читатель байткода в JIT и дизассемблер, чтобы трое не разошлись в показаниях, резервирование ячеек в метаданных опкодов компилятора, и запись содержимого кэша по одному байту на ячейку, с байтом опкода, оставленным равным `CACHED`. Последнее — самое неочевидное: CPython обходит объект кода по одной кодовой единице и не может знать, что статический опкод резервирует единицы за собой, поэтому сырое 32-битное значение, первый байт которого случайно называет опкод с собственным кэшем, отправляет `deopt_code` за пределы объекта кода. `INVOKE_INDIRECT_CACHED` намеренно оставлен неспециализированным: кэширование `PyObject**` стоило бы восьми единиц на каждом месте `INVOKE_FUNCTION` ради редкого контейнера, который бессмертен, но не неизменяем. Патчи лежат в приложенном дереве.

Проверка здесь важнее самой правки, потому что интерпретатор, который молча не специализирует, читался бы как нулевой результат, а не как ошибка. Собственный набор CinderX `test_compiler/test_static`, 3910 тестов, проходит под `PYTHONMALLOC=debug` в обеих конфигурациях, до правки неадаптивная сборка этот набор *аварийно завершала*, а адаптивная не могла исполнить Static Python вообще. Разведочный проход набора теперь докладывает три строки и ни одного отказа проверки в обеих конфигурациях при одной и той же эталонной контрольной сумме, а отдельная проба читает `_co_code_adaptive` после исполнения и требует, чтобы кэшированный опкод там присутствовал, — то есть показано, что специализации срабатывают, а не просто никому не мешают.

Результат: Static Python — вот где замысел окупается, и только с JIT. На целочисленном произведении матриц 96×96 тот же исходник, скомпилированный как Static Python с распакованным `Array[int64]`, занимает 716.1 мс под интерпретатором CinderX против 70.4 мс у обычного Python с объектами на том же интерпретаторе — *замедление* в 10.2 раза. Инспекция байткода подтверждает понижение уровня: 61 из 158 инструкций

— примитивные или в форме `extended-opcode`. Скомпилированная, та же функция занимает 4.25 мс в большинстве рабочих процессов — и 6.4 мс в остальных, о чём нужно сказать прежде, чем выводить из неё какое-либо отношение.

Эта строка — единственное место в работе, где размещение адресного пространства добирается до заголовочного числа. Её отсчёты бимодальны: 4.25 мс либо 6.4 мс, между ними ничего, и режим оказывается свойством процесса, а не отдельного отсчёта: из шестидесяти отсчётов двух конфигураций сборки 33 попали в быстрый режим и 27 в медленный, причём внутри процесса все отсчёты согласны между собой. Медиана здесь поэтому не является статистикой — она сообщает тот режим, который выиграл жеребьёвку, и на одинаковых бинарниках вышла 4.25 мс в одной конфигурации и 6.43 мс в другой. Отключение рандомизации схлопывает эффект: двенадцать прогонов под `setarch -R` все дают 4.25–4.26 мс. Больше так не ведёт себя ничто в наборе, включая то же ядро с объектами под тем же JIT, у которого тридцать отсчётов укладываются в $1.02\times$: к размещению чувствителен именно плотный скомпилированный цикл. Это и есть то, что даёт включённый ASLR (§3): замер с фиксированным размещением сообщил бы один из двух режимов как ответ, и о существовании второго никто бы не узнал.

С чем тогда сравнивать эти 4.25 мс (быстрая раскладка; отсчёт взят из адаптивного прогона, а компилируемую строку адаптивный флаг не двигает — это показано следующим абзацем)? «В 16.6 раза быстрее Python с объектами» — сравнение, напрашивающееся первым, и оно записывает на счёт типов то, что сделал компилятор, потому что вторая половина сравнения интерпретируется. То же ядро с объектами под тем же JIT занимает 48.4 мс, значит, сам JIT стоит здесь $1.5\times$, а *типы поверх него* — $11.4\times$, либо $7.5\times$ для процесса, которому досталось медленное размещение. Это и есть число, нужное тому, кто внедряет проект, — и это диапазон, а не точка.

Адаптивный путь, теперь когда он работает, стоит большой доли интерпретируемой цены и нисколько — компилируемой: статическая интерпретация падает с 716.1 мс до 410.7 мс, экономия 43 %, а скомпилированная строка не двигается. Форма ожидаемая: флаг специализирует диспетчеризацию в интерпретаторе, а JIT не диспетчеризует, — и она же решает вопрос о том, зачем примитивы нужны. Даже полностью специализированные, в интерпретации они в 5.8 раза медленнее, чем если бы их не было, — и в 8.5 раза медленнее, чем если тот же нетипизированный исходник отдать JIT, — и причина видна в кодировании: каждая примитивная операция — это префикс `EXTENDED_OPCODE` плюс статический опкод, поэтому она стоит двух косвенных диспетчеризаций там, где обычному опкоду хватает одной: сначала собственный `computed-goto` переход интерпретатора в обработчик префикса, затем второй переход по таблице из 103 входов, общей для всех статических опкодов. Дизассемблирование собранного интерпретатора показывает эту вторую диспетчеризацию единой точкой перехода — внутри схемы `computed goto`, весь смысл которой в том, чтобы у каждого опкода была своя точка. Это свойство порта на 3.14, а не замысла: интерпретатор 3.12 в том же дереве исходников не имеет `EXTENDED_OPCODE` вовсе и добирается до `LOAD_LOCAL` или `PRIMITIVE_BINARY_OP` через главную таблицу диспетчеризации, как до любой другой инструкции. Однако причина глубже: интерпретируемый путь вообще не реализует примитивы как примитивы. В интерпретаторе 3.14 значение `int64` лежит на стеке операндов и во фрейме обычным `PyLong`: `PRIMITIVE_BINARY_OP` принимает два `PyObject*`, распаковывает их и аллоцирует под результат новый `PyLong`, `STORE_LOCAL` на каждом исполнении заново разрешает объявленный примитивный тип через загрузчик классов и снова упаковывает значение, `LOAD_LOCAL` получает индекс слота, конвертируя питоновское целое из `co_consts`. Значит, типизированная форма платит ровно ту аллокацию, ради устранения которой существует, и добавляет сверху разрешение типа и вторую диспетчеризацию — отсюда же и 43 %, которые даёт адаптивный путь: именно его кэши снимают разрешение типа на каждой записи. В прямом измерении одно примитивное `s = s + i` на `int64` стоит 111 нс против 28 нс у того же оператора на обычных целых Python на том же интерпретаторе. Единственный потребитель, который держит `int64` в регистре, — это JIT, и отсюда практический вывод: примитивные опкоды — это запись для компилятора. В интерпретации они выполняют ту же упаковку, что и обычные целые Python, так что обходятся вчетверо дороже кода, который заменяют.

Задержку компиляции самого JIT стоит записать рядом с Numba: CinderX компилировал каждую из восьми функций ядер за 1.1–5.2 мс, то есть в 41–194 раза дешевле, чем 215 мс у Numba на одну сигнатуру (§4.2). У рантаймового JIT, который платит несколько миллисекунд за функцию, точки окупаемости фактически нет — и поэтому выбор в его пользу или против него определяется не разогревом, а установившимся поведением на нетипизированном коде.

Результат: цену сборки решает не число потоков. Параллельный сборщик — единственная возможность CinderX, у которой собственная настройка оказалась не той переменной, и причину видно в исходнике. На рабочих потоках выполняется только `deduce_unreachable` — вычитание ссылок и разметка достижимости, финализаторы и освобождение остаются на одном, так что какую долю сборки занимают эти две фазы, такую долю опция и способна затронуть. Вдобавок простаивающий размечающий воркер не засыпает — `ci_gc_steal_backoff` выполняет активное ожидание в холостом цикле, — поэтому лишние потоки превращаются в помеху, как только появляется живой набор, который надо разметить.

Двигает же число то, что сборщику вообще видно, — а это форк-сервер меняет намеренно. Рисунок 8 (снизу справа) измеряет форму одного воркера: долгоживущее состояние из $8 \cdot 10^5$ объектов плюс $8 \cdot 10^5$ объектов мусора на запрос, собираемые при видимом состоянии, после `gc.freeze()` и после `immortalize_heap()` из CinderX — который выполняет полную сборку, вызывает `freeze` и лишь затем увековечивает выживших, чтобы их счётчики ссылок перестали пачкать страницы после форка. При видимом состоянии сборка занимает 187 мс, а параллельный сборщик доводит её до 276 мс (в 1.48 раза медленнее, $t = -139.2$). Как только состояние выведено из поля зрения сборщика, сборка занимает 129 мс — в 1.45 раза быстрее, чем если оставить его внутри ($t = 161.9$), — и лишь тогда параллельная опция вообще уходит в плюс, давая $1.06 \times$ ($t = 13.7$).

Отсюда следует двоякое. Рычагом цены сборки служит состав кучи, и стоит он 1.45 раза там, где число потоков стоит 1.06 раза в единственной конфигурации, где оно не делает хуже. И рантайм CinderX сам по себе цену сборки не меняет: обычный CPython 3.14.6 собирает ту же кучу за 186 мс против 187 мс у CinderX, и это различие критерий значимости разрешить отказывается ($1.004 \times$, $t = -1.33$). Чего бы этот рантайм ни стоил, дело не в том, что у него быстрее сборщик.

Результат: чего стоило до этого добаться. CinderX собрался и заработал на этом хосте нативно в обеих конфигурациях, с нулём ошибок компиляции в каждой: все восемь функций ядер принудительно скомпилировались и дали верные результаты, а Static Python отработал целиком в обеих. Что потребовалось сборке, зафиксировано в скриптах сборки и журналах в артефакте, которые и являются источником этого абзаца и не являются измерениями: clang обязателен и используется и для форка, и для обычной эталонной сборки, и для расширения, каждой конфигурации нужно собственное дерево исходников, а загрузчик Static Python молча выдаёт байткод с объектами, если не вызвать `cinderx.compiler.strict.loader.install()` — вызов описан только в руководстве Static Python, не в справочнике API, и именно он отделяет измерение Static Python от измерения обычного байткода под другим именем.

Три ловушки лежат не в сборке, а в самом измерении, и каждая выдаёт число, а не ошибку. Компиляция ядра до первого вызова — самое очевидное решение и то, к которому подталкивает `force_compile`, — компилирует неспециализированный байткод: на тех же шести ядрах этот путь даёт $0.85\text{--}1.53 \times$ обычной сборки там, где собственный порог JIT даёт $0.33\text{--}0.79$, то есть меняет знак результата. Вызов `force_compile` на функции, которая уже исполнялась, порождает скомпилированный код, в который никто не заходит, а `is_jit_compiled` при этом отвечает `True`, хотя все вызовы идут через интерпретатор. И в обратную сторону `is_jit_compiled` свидетель тоже ненадёжный: при включённом HIR-инлайнере — а он включён по умолчанию — он отвечает `False` про функцию, которую JIT исполняет. Из трёх признаков только `count_interpreted_calls` говорит, что именно исполняется, — именно на нём набор и ставит проверку. Четвёртая ловушка стоит рядом с ними и к JIT отношения не имеет: `ruperf` намеренно вычищает окружение рабочих процессов — так бенчмарк перестаёт зависеть от той оболочки, из которой его запустили, — а JIT читает свою конфигурацию из окружения. Значит, конфигурация, заданная для прогона, доходит до ведущего процесса и не доходит до рабочих, а меряют именно они. Наш первый проход по опциям выше девять раз измерил конфигурацию по умолчанию, успешно и без единого предупреждения. Теперь набор объявляет ожидаемую настройку в командной строке, которую `ruperf` рабочим пересобирает, и падает в любом процессе, куда она не дошла.

Одну опцию пришлось починить, прежде чем её вообще стало можно измерить: `-x cinderx-jit-stable-frame=0`, консервативный режим для кода, для которого допущение о стабильности неверно, падал с сегфолтом на девятистрочном примере. Правка на две функции лежит в артефакте, и с ней собственный статический набор тестов CinderX отработывает в этом режиме целиком. Заодно эта опция молча отключает инлайнер HIR, о чём подсказка к ней не говорит.

Вывод. Слово «экспериментальный» этот проект описывает плохо, и таблица 7 заменяет её отчётом по возможностям. Cinder/CinderX не нестабилен: он собирается в обеих конфигурациях с нулём ошибок компиляции, он работает, и его JIT корректен. Бесплатным он при этом не является, а измерить его наоборот легко. На шести наших нетипизированных ядрах его рантайм стоит $1\text{--}33\%$ в любой из конфигураций: поставленный им вычислитель фреймов заменяет тот цикл, который CPython специализирует. А его JIT эту цену с запасом возвращает, исполняя те же ядра за $0.33\text{--}0.79 \times$ обычной сборки, то есть быстрее в $1.3\text{--}3.1$ раза. Из трёх функций рантайма, которые мы смогли задействовать, одна даёт крупный выигрыш по своей оси (ленивые импорты), одна не дала на наших пробах ничего (облегчённые фреймы: в пределах 3 % на голом форке и на $13\text{--}37\%$ дороже под рантаймом CinderX), а третья включает переменную, которой её собственная настройка не является (параллельный сборщик: он в 1.48 раза медленнее, пока долгоживущая куча ему видна, и в 1.06 раза быстрее, когда иммортализация убрала её из поля зрения). Static Python добавляет поверх JIT ещё $11.4 \times$ на типизированном ядре, или $7.5 \times$ на медленном режиме размещения и требует переписывания на примитивных типах и последовательности инициализации, описанной только в руководстве, — а примитивы, которых он просит, стоят того лишь потому, что их кто-то компилирует, в интерпретации, даже со специализирующим путём, который нам пришлось дописать, они в 5.8 раза медленнее того кода с объектами, который заменяют. Заслуженная проектом запись поэтому

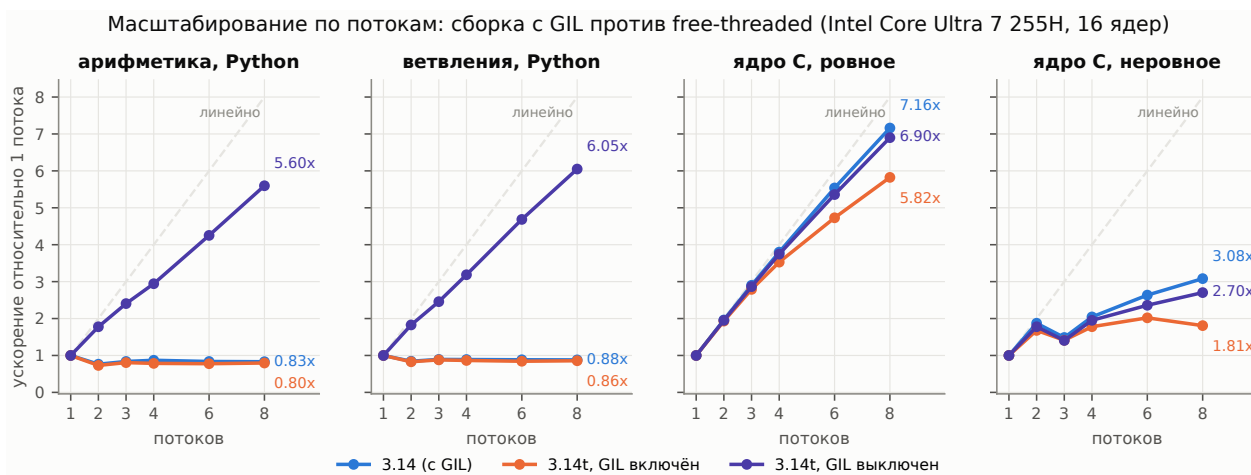


Рис. 10: B5: масштабирование по потокам. Ускорение приведено относительно той же конфигурации на одном потоке, пунктир — линейное масштабирование.

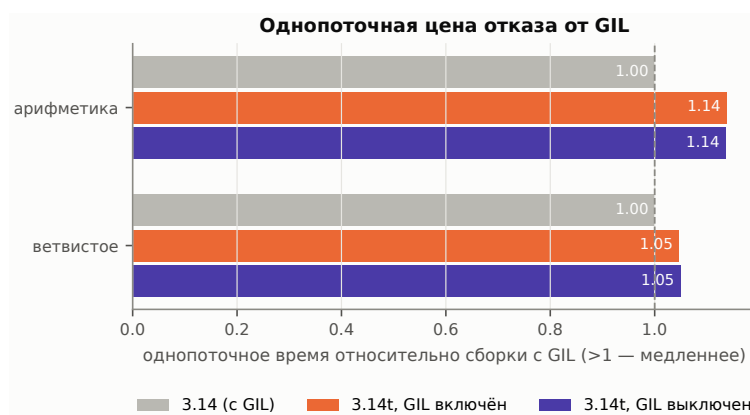


Рис. 11: B5: чего стоит отказ от GIL, когда работает всего один поток.

шире, чем показал наш первый замер, и по-прежнему точна: JIT стоит того на обычном коде, если позволить ему компилировать после того, как интерпретатор специализировал байткод, ленивые импорты окупаются там, где сервис импортирует заметно больше, чем исполняет — здесь 0 тел модулей из 200, и ровно ничего там, где тела исполняются, — а отдача в порядок величины принадлежит командам, способным переписать горячий код типизированными примитивами и поддерживать форк. Чего наши измерения не покрывают и что должна покрывать оценка проекта — это та нагрузка, ради которой он создавался: долгоживущий серверный код, где машинария фреймов, от которой мы не увидели эффекта, и должна иметь значение и где состав кучи, решающий судьбу сборщика, складывается по замыслу, а не строится, как здесь, ради замера.

8 B5: насколько масштабируется free-threaded сборка?

Вопрос. Free-threaded сборка [4] снимает блокировку интерпретатора. Какое ускорение это даёт вычислительной нагрузке по мере роста числа потоков и какую цену платит единственный поток?

Постановка. Три конфигурации из одного исходника 3.14.6: сборка по умолчанию (с GIL), free-threaded сборка с `PYTHON_GIL=1` (GIL включён обратно — это изолирует цену самой free-threaded сборки от эффекта снятия блокировки) и free-threaded сборка как задумано. Три вида нагрузки: арифметика на чистом Python, ветвистая классификация байтов на чистом Python и ядро на C через `stypes` (которое освобождает GIL, поэтому ожидается масштабирование во всех конфигурациях). Общий объём работы постоянен и делится на $T \in \{1, 2, 3, 4, 6, 8\}$ потоков, для сравнения измеряется и вариант с пулом процессов. Этот набор — единственное исключение из политики привязки в работе: все прочие наборы привязаны к шести производительным ядрам, а свип по потокам, привязанный туда же, упёрся бы в шесть и изготовил бы полку, — поэтому он идёт на процессорах 0–13, то есть на шести производительных и восьми энергоэффективных ядрах.

Результат: масштабированию мешает блокировка, и её снятие работает. Оба участвующих интерпретатора собраны из одного архива 3.14.6 одним скриптом и одним компилятором, а их `CONFIG_ARGS` различаются ровно одним токеном, `--disable-gil`. Третья конфигурация — та же `free-threaded` сборка с `PYTHON_GIL=1`, что возвращает блокировку, не меняя бинарник. Поэтому сравнение изолирует блокировку, затем сборку и никогда — упаковку.

Рисунок 10. Все три конфигурации измерялись подряд, и их машинные пробы согласуются с точностью до 0.25% (4.069, 4.059 и 4.062 мс), так что сравнение между ними не стоит на дрейфе, контрольный однопоточный повтор сместился не более чем на 1.03 %.

Арифметика на чистом Python под GIL не масштабируется — ей становится хуже: $0.76\times$ на двух потоках и $0.83\times$ на восьми, — и, что важно, она ведёт себя так же на `free-threaded` сборке, когда GIL возвращают через `PYTHON_GIL=1` ($0.73\times$ и $0.80\times$), и именно это делает ответственной блокировку, а не сборку. Со снятой блокировкой та же нагрузка достигает $1.78\times$ на 2 потоках, $2.94\times$ на 4, $4.25\times$ на 6 и $5.60\times$ на 8, ветвистая нагрузка достигает $1.83\times$, $3.19\times$, $4.69\times$ и $6.05\times$. Масштабирование настоящее и сублинейное с первого же шага: эффективность на поток для арифметической нагрузки составляет 0.89 на двух потоках, 0.74 на четырёх и 0.70 на восьми, а восьмой поток находится на энергоэффективном ядре — свипу доступно шесть производительных ядер, и остальное он берёт из энергоэффективного класса.

Контроль на нативном коде делает механизм явным. Сбалансированное ядро на C, вызываемое через `ctypes`, масштабируется во всех конфигурациях — $7.16\times$ под GIL, $5.82\times$ на `free-threaded` сборке с включённым GIL, $6.90\times$ с выключенным, — потому что `ctypes` освобождает блокировку на время вызова. Если горячий цикл уже на C, снятие GIL не даёт ничего, блокировка сериализует исполнение байткода. Намеренно несбалансированный вариант того же ядра дотягивает лишь до $1.81\text{--}3.08\times$ на восьми потоках независимо от конфигурации и даже не монотонен по числу потоков — все три конфигурации на трёх потоках медленнее, чем на двух, а `free-threaded` сборка с включённым GIL лучше всего идёт на шести потоках ($2.02\times$) и откатывается к $1.81\times$ на восьми, — напоминание, что кривую масштабирования можно испортить одним лишь разбиением работы и что сообщённое « $N\times$ на потоках» говорит о разбиении не меньше, чем о рантайме.

Для сравнения: та же арифметика на восьми процессах на обычной сборке даёт $5.62\times$ — на той же базе, что и рисунок 10, то есть по более быстрому из двух однопоточных измерений этой конфигурации, — при уже разогретом пуле, так что в счёт идут только отправка, вычисление и сбор. Это число и $5.60\times$ у отказа от GIL несопоставимы напрямую: у каждого своя база — однопоточное время своей же сборки, — а `free-threaded` сборка стартует на 11 % позади (355.7 против 320.3 мс на однопоточной базе этого свипа, следующий абзац измеряет ту же цену как $1.14\times$ на другой, куда более короткой нагрузке). В абсолютном времени восемь процессов заканчивают за 57.0 мс против 63.6 мс у восьми `free-threaded` потоков, то есть в 1.115 раза быстрее ($t = -34.3$, значимо), а не наравне. Контроль, отделяющий сборку от механизма, лежит в том же наборе: восемь процессов на `free-threaded` сборке занимают 64.4 мс, так что против её же восьми потоков на 63.6 мс потоки выигрывают 1.3 % ($t = 2.67$, значимо). Процессы здесь не обыгрывают потоки — обыгрывает однопоточная цена `free-threaded` сборки, и она начисляется обеим её кривым. Чего отказ от GIL избегает — так это дублирования адресного пространства и сериализации аргументов и результатов, потокам они не нужны вовсе, а процессам эта нагрузка — четыре целых на вход, одно на выход, при разогретом пуле — даёт самую дешёвую их версию из возможных.

Чего стоит отказ от GIL. Рисунок 11. В один поток `free-threaded` сборка в 1.14 раза медленнее на арифметике (14.26 против 16.20 мс) и в 1.05 раза медленнее на ветвистом ядре (25.23 против 26.47 мс). Возвращение GIL на том же бинарнике этого не отыгрывает — $1.14\times$ и $1.05\times$, те же две цифры до двух знаков, — что помещает цену в сборку, а не в отсутствие блокировки: `free-threaded` интерпретатор платит за другое размещение объектов и другую схему подсчёта ссылок независимо от того, есть блокировка или нет. На реалистичном смешанном конвейере §10 та же цена составляет 0.4 %, так что эти два микротеста — пессимистичный её край, и второй поток её возвращает в любом случае.

Вывод. Отказ от GIL даёт результат, и даёт примерно то, что позволяют ядра: $5.60\times$ на арифметике и $6.05\times$ на насыщенном ветвлениями коде при восьми потоках, тогда как обычная сборка даёт $0.83\times$ и $0.88\times$, — то есть на обычной сборке добавление потоков к этой нагрузке делает её медленнее. Именно контроль `PYTHON_GIL=1` на том же бинарнике позволяет отнести это к блокировке, а не к сборке. Рядом стоят три записи. Первая — цена: $1.05\text{--}1.14\times$ однопоточного замедления на этих микротестах и 0.4 % на смешанном конвейере, и это цена сборки, а не блокировки — тот же бинарник с включённой обратно блокировкой платит её тоже, — поэтому она начисляется каждой части программы, включая те, которые ни одного потока не запускают, второй поток её возвращает. Вторая запись — предусловие: если горячий цикл уже нативный, выигрывать нечего, потому что `ctypes` освобождает блокировку и то же ядро масштабируется в 7.16 раза на обычной сборке. Третья — что восемь процессов на обычной сборке заканчивают эту нагрузку за 57.0 мс против 63.6 мс у восьми `free-threaded` потоков, но это сравнение двух интерпретаторов, а не двух механизмов: внутри одной и той же `free-threaded` сборки потоки (63.6 мс) быстрее процессов (64.4 мс), и весь разрыв даёт её однопоточная цена. К тому же задача

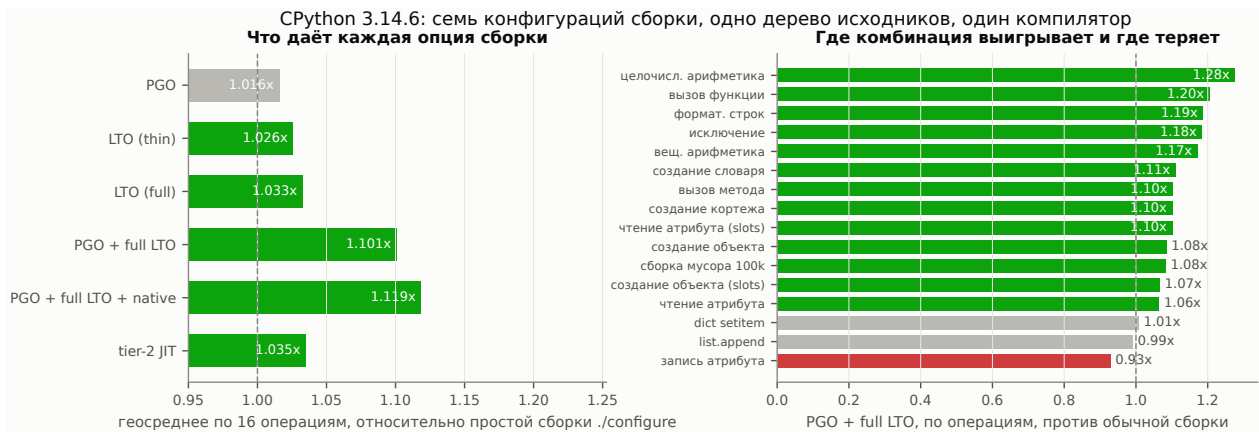


Рис. 12: В6: семь конфигураций сборки одного дерева исходников CPython 3.14.6 одним компилятором. Слева — геометрическое среднее по набору операций. Справа — где PGO с full LTO выигрывает и где проигрывает.

передаёт по четыре целых на вход и одно на выход, — сериализация, которой потокам не нужно вовсе, обходится здесь процессам дешевле, чем на любой другой нагрузке.

9 В6: сколько в PGO, LTO и -march=native?

Вопрос. Интерпретатор сам является программой на C, поэтому к нему применимы оптимизация по профилю, оптимизация на этапе компоновки и архитектурно-специфичные флаги. Если зафиксировать исходники, компилятор и всё остальное, сколько даёт каждая из них и складываются ли они?

Постановка. Семь конфигураций, все из одного дерева исходников CPython 3.14.6, вложенного в артефакт, все собранные одним и тем же `/usr/bin/clang` и различающиеся только строкой `configure`: простой `./configure`, `--enable-optimizations` (PGO), `--with-lto=thin`, `--with-lto=full`, PGO вместе с full LTO, то же с `CFLAGS=-march=native`, и `--enable-experimental-jit` (§9.1). Различие между `thin` и `full` стоит проговорить, потому что его легко потерять: опция CPython выглядит как `--with-lto=[full|thin|no|yes]`, а голый `--with-lto` на этой платформе приводится к `-flto=thin`, так что «PGO + LTO» в обычной записи означает PGO с `thin` LTO. Каждая конфигурация прогоняет набор операций рантайма из §6 под `ruperf`, а числа ниже — геометрические средние по его 16 операциям относительно простой сборки.

Результат: полезной записью оказывается комбинация, а не одиночная опция. Рисунок 12, левая панель, геометрическое среднее по 16 операциям относительно простой сборки `./configure` того же дерева:

- **PGO отдельно:** $1.016\times$ — наименьшая из одиночных опций, и она неоднородна: она стоит $1.131\times$ на форматировании строк и $1.111\times$ на целочисленной арифметике, обходясь в 5 % на чтении атрибута ($0.946\times$, $t = -12.5$, значимо).
- **LTO отдельно:** $1.026\times$ для `thin` и $1.033\times$ для `full` — и, в отличие от PGO, обе положительны почти на каждой операции. Это не одна и та же сборка: full LTO опережает thin на целочисленной арифметике ($1.0114\times$, $t = 3.35$), на вызовах методов ($1.0347\times$, $t = 13.8$) и на форматировании строк ($1.0254\times$, $t = 9.65$) — все значимо. Запас составляет от одного до трёх с половиной процентов, так что thin LTO даёт большую часть того, чего стоит full LTO, при существенно меньших затратах на компоновку.
- **PGO вместе с full LTO:** $1.101\times$ — больше, чем произведение двух опций ($1.016 \times 1.033 = 1.050$), так что именно здесь уровень сборки и окупается. Эффект к тому же широкий: четырнадцать операций из шестнадцати улучшаются, во главе с целочисленной арифметикой $1.276\times$, вызовами функций $1.204\times$, форматированием строк $1.187\times$, проходом исключения $1.184\times$ и вещественной арифметикой $1.172\times$. Хуже становится только запись атрибута ($0.928\times$), а `list.append` ($0.990\times$, $t = -3.86$) и присваивание в dict ($1.007\times$, $t = 4.21$) сдвигаются менее чем на 1%, оба различия критерий разрешает.
- **Добавление -march=native сверху:** $1.119\times$ — ещё 1.6 % и лучшая измеренная конфигурация. От той же сделки она не свободна: запись атрибута падает дальше, до $0.902\times$.

Складывание — содержательный результат этого раздела, и оно противоположно тому, что подсказывают числа по отдельным опциям: PGO сам по себе стоит 1.6 %, full LTO сам по себе — 3.3 %, а вдвоём они дают

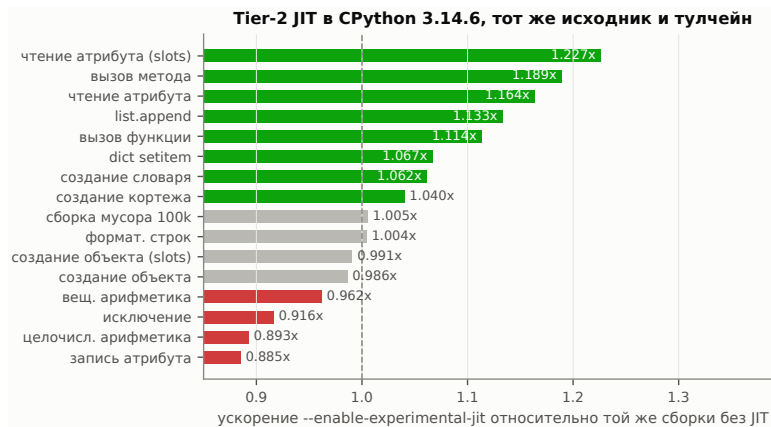


Рис. 13: B66: `--enable-experimental-jit` против такой же сборки без него, по операциям. Тот же исходник, тот же компилятор, тот же набор.

10.1 %. Рекомендация, собранная из измерений отдельных опций, поставила бы эту строку последней, измеренная как конфигурация сборки, а не как список флагов, она стоит больше, чем выпуск работы над интерпретатором (§6). Правдоподобный механизм в том, что профиль позволяет межпроцедурному встраиванию выбирать с пользой по всему вычислительному циклу вместо встраивания по эвристикам размера, — ровно тот случай, который ни одна из опций не может обеспечить в одиночку, мы приводим эффект и профиль по операциям, который делает его видимым, не называя ответственный проход оптимизатора. Единственная операция, сопротивляющаяся ему в каждой конфигурации, — запись атрибута, $0.88\text{--}0.98\times$ во всех шести, — это то, что стоит проверить на любой программе, где она доминирует.

Задержка запуска движется здесь в ту же сторону и слабее: запуск с импортом стандартной библиотеки падает с 69.4 мс на простой сборке до 66.2 мс с PGO и 65.6 мс с PGO и full LTO — с `-march=native` или без него. Значит, конфигурация, окупающаяся в установившемся режиме, на этой сборке не платит за это на запуске, — но запуск измеряется как полное время процесса для каждой из тех же семи сборок именно потому, что это не то, что стоит предполагать.

Вывод. Уровень сборки даёт реальный выигрыш, он крупнее, чем выглядит по отдельным опциям, и достаётся не тем, о чём обычно думают. PGO отдельно стоит 1.6 %, LTO отдельно — 2.6–3.3 %, окупается же комбинация, на 10.1 %, поднимающихся до 11.9 % с `-march=native`, — и это единственное место в работе, где архитектурный флаг даёт устойчивый (пусть и малый) выигрыш. На фоне $3.2\text{--}42.8\times$, доступных уровнем выше (альтернативный рантайм, §11), или $32\text{--}137\times$ двумя уровнями выше (компилируемые ядра, §4), 12 % на уровне сборки — всё ещё оптимизация последней мили: её стоит взять, когда интерпретатор ваш, и никогда не стоит ставить в приоритет, — но теперь её стоит брать целиком, а не как список опций, и стоит измерить на коде, насыщенном записью атрибутов, прежде чем отгружать.

9.1 B66: помогает ли уже собственный экспериментальный JIT CPython?

Постановка. Собственный tier-2 JIT CPython [11], экспериментальный с 3.13, регулярно упоминают среди доступных вариантов и почти никогда не измеряют. Его сборка — седьмая конфигурация §9: тот же исходник и тот же компилятор, что у простой сборки, а единственное отличие — `--enable-experimental-jit`. Для сборки дополнительно нужен LLVM 19 для генератора трафаретов, это документированное требование самой функции и единственная причина, по которой во всём артефакте появляется второй тулчейн.

Результат: tier-2 JIT улучшает больше операций, чем замедляет. Рисунок 13. Сборка с JIT даёт геометрическое среднее $1.035\times$ от простой сборки — примерно на 3.5% быстрее. Десять операций из шестнадцати улучшаются, и улучшения приходятся туда, где трассирующий tier-2 оптимизатор и должен помогать: чтение атрибута у экземпляра со слотами $1.227\times$, вызовы связанного метода $1.189\times$, чтение атрибута $1.164\times$, `list.append` $1.133\times$, вызовы функций $1.114\times$, присваивание в dict $1.067\times$. Шесть операций становятся хуже. Крупнейшая из них — запись атрибута, $0.885\times$, а следом идёт та, которую труднее всего объяснить: целочисленная арифметика, самый дружелюбный к JIT случай набора, — $0.893\times$. Проход исключения — $0.916\times$, вещественная арифметика — $0.962\times$, а создание экземпляра сдвигается менее чем на 1.5 % в обе стороны ($0.986\times$ у обычных объектов и $0.991\times$ у слотовых), и оба различия критерий разрешает. Запуск не изменился (69.4 мс на импорт стандартной библиотеки против 69.4 мс у простой сборки), так что за JIT на этом наборе не платят при старте

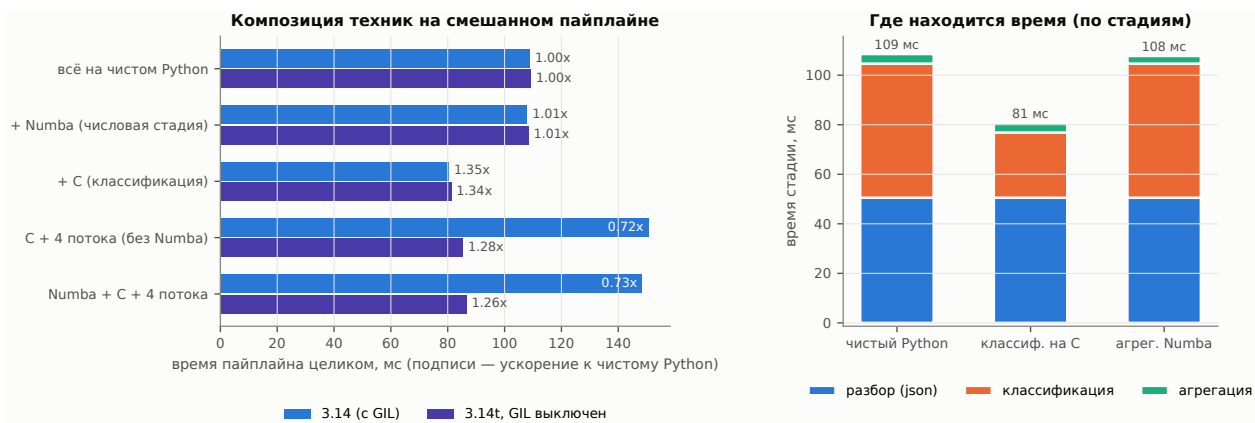


Рис. 14: В7: наложение техник на смешанном конвейере (слева) и где находится время по стадиям (справа).

процесса. Сборка настоящая: `_Py_JIT` и `_Py_TIER2` присутствуют в `PY_CORE_CFLAGS`, а `sys._jit.is_available()` и `sys._jit.is_enabled()` обе возвращают истину, — то есть это работающий JIT, а не JIT, который не включился.

Вывод. Осторожное «да» с приложенным числом и с оговоркой внутри. На этом хосте и в этой версии собственный JIT CPython стоит около 3.5% на операциях уровня интерпретатора — того же порядка, что и конкурирующий с ним уровень сборки, и достаётся переключением одного ключа `configure`, а не прогоном обучающего профиля. Оговорка — в распределении, а не в среднем: он окупается на путях доступа к атрибутам и вызовов до 23 % и теряет 11 % на целочисленной арифметике, так что по какую сторону среднего окажется программа, зависит от того, что она делает. Он по-прежнему справедливо помечен как экспериментальный, наш набор имеет форму микротестов, а не трасс, — но строка уже не пуста.

10 В7: складываются ли техники?

Вопрос. Если ни один инструмент не выигрывает всюду, практическим ответом становится композиция: JIT для численной части, нативный код для нерегулярной, оптимизированный рантайм под ними, потоки для масштаба. Складываются ли такая композиция на реалистичном смешанном конвейере?

Постановка. Обыкновенный смешанный конвейер на 20 000 JSON-записях: разбор модулем `json` из стандартной библиотеки, классификация строкового поля по каждой записи (ветвистая), свёртка числовых полей (вычислительная; стадия *агрегация* на рисунке 14), выдача сводки. Четыре варианта добавляют по одной технике за раз, и каждый обязан выдать побайтово ту же сводку. Весь набор прогоняется дважды: один раз на обычной сборке 3.14.6 и один раз на `free-threaded`.

Результат: какую технику стоит применять, решает профиль стадий. Рисунок 14. Конвейер на чистом Python занимает 108.9 мс на обычной сборке, которые распределены как 50.5 мс разбора, 54.1 мс классификации и 3.9 мс числовой свёртки. Наложение техник по одной даёт:

- **Numba на числовой стадии:** $1.01\times$ по конвейеру в целом. Сама стадия улучшается с 3.87 до 3.10 мс, но она составляет 3.6% конвейера, поэтому выигрыш исчезает — измеренный закон Амдала. Стадия стоила 3.6% прогона ещё до того, как к ней применили технику, и это число было известно заранее.
- **C на стадии классификации:** $1.35\times$ по конвейеру в целом. Стадия улучшается в 2.06 раза (54.1 → 26.3 мс) и составляет 50% конвейера. Сюда входят 20 000 пересечений `stypes` по 166 нс (≈ 3.3 мс, §5).
- **Четыре потока на сборке с GIL:** $0.72\times$ — регрессия. Оба потоковых варианта *медленнее* последовательного (150.7 и 148.5 мс против 108.9 мс). Цикл по записям — это байткод уровня Python, поэтому GIL его сериализует, а нарезка на части и управление потоками остаются чистыми накладными расходами.
- **Четыре потока на free-threaded сборке:** $1.28\times$ — настоящий выигрыш, которого всё равно не хватает. Тот же код падает со 150.7 до 85.4 мс, то есть снятие блокировки превращает регрессию в 38 % в улучшение на 28 %. Но последовательный вариант «Numba плюс C» всё равно быстрее, на обеих сборках (80.5 мс с GIL, 81.3 мс без него).



Рис. 15: B8: PyPy 3.11 против CPython 3.14.6. Слева — шестнадцать операций набора из §6, который PyPy исполняет без изменений. Справа — шесть ядер, с неизменёнными исходниками на чистом Python.

Полностью скомпонованный вариант — Numba, C и четыре потока — теперь идёт на обеих сборках, потому что Numba устанавливается и импортируется на free-threaded интерпретаторе. На сборке с GIL он даёт $0.73\times$, на free-threaded — $1.26\times$, то есть ниже того, чего две из трёх его техник достигают без третьей. Лучшая измеренная где-либо в этом разделе конфигурация — Numba плюс C, последовательно, на обычной сборке: 80.5 мс, $1.35\times$. Free-threaded сборка стартует на этом конвейере с отставанием всего в 1.0035 раза, так что сравнение решает не её штраф, а разбиение работы.

Вывод. Композиция — верный принцип: лучшая измеренная нами конфигурация действительно объединяет две техники и даёт $1.35\times$ относительно базы на чистом Python, — но она не аддитивна, и рекомендация обязана это сказать. На этом конвейере две из четырёх техник дали ничего ($1.01\times$) или меньше, чем ничего ($0.72\times$ на обычной сборке), и какие именно провалились, определил профиль стадий, а не технология: потокам нужно делить стадию, которая ещё не является самым быстрым местом конвейера, а здесь им досталась стадия, уже перенесённая в C. Практическая форма правила поэтому такая: профилировать, применить технику, подходящую доминирующей стадии, и *профилировать снова* — потому что после переноса классификации в C разбор стал занимать 50.5 мс из оставшихся 80.5 мс, и ни JIT, ни потоки его не касаются.

11 B8: где на тех же ядрах находится PyPy?

Вопрос. Каждая технология из B1 и B2 требует правки кода. PyPy не требует никакой: это трассирующий JIT для обычного байткода Python [6]. Что он даёт на тех же ядрах без единой правки исходника и чего это стоит?

Постановка. PyPy 3.11 (v7.3.20, реализует Python 3.11.13) исполняет наши ядра и набор операций без изменений на том же хосте, под rperf, ровно так же, как это делает каждая сборка CPython. NumPy, Cython, pybind11, ctypes и cdylib на Rust там работают и измеряются. Единственное исключение — Numba: ей нужен llvmlite, для которого под PyPy не публикуют wheel-пакетов и который под PyPy не собирается, так что набор записывает её строки как unavailable с ModuleNotFoundError, а не как медленный результат.

Результат: широкое распределение вокруг большого геометрического среднего. На наборе операций из §6, выполненном на PyPy без изменений (рисунок 15, слева), PyPy в среднем геометрически в $7.3\times$ быстрее CPython 3.14, и распределение чрезвычайно широкое: проходы исключения $87.7\times$, целочисленная арифметика $47.4\times$, вызовы функций $37.9\times$, вызовы связанного метода $24.9\times$, чтение атрибута $19.6\times$ — и на другом конце построение dict-литерала $0.86\times$, где PyPy из двух медленнее, создание кортежа $1.39\times$, сборка циклического мусора $1.81\times$ и форматирование строк $2.19\times$. На шести ядрах, с неизменёнными исходниками (рисунок 15, справа), PyPy даёт $42.8\times$ (matmul), $23.1\times$ (мандельброт), $19.3\times$ (сумма массива), $7.5\times$ (токенизация), $5.8\times$ (бинарные деревья) и $3.2\times$ (BFS).

Три следствия скорее переупорядочивают таблицу решений, чем добавляют к ней строку.

Во-первых, **в вычислительном классе PyPy играет в той же лиге, что компилируемые реализации, и стоит нулевых изменений кода.** Cython, Numba, C и Rust достигали на этих ядрах $32\text{--}137\times$ (§4) — но каждый требовал либо аннотаций типов, либо второго языка, либо шага сборки. PyPy достиг $19.3\text{--}42.8\times$ на тех же

файлах вообще без правок: $19.3\times$ против компилируемой полосы $32\times$ на сумме массива и $42.8\times$ против $72-137\times$ на произведении матриц. Это тот же порядок величины за отсутствие работы, и для кодовой базы, которой нельзя обраться компилируемым компонентом, это самый дешёвый крупный множитель во всей работе.

Во-вторых, **компилируемая экосистема на PyPy не закрыта — но идущие через C-API пути стоят порядков величины.** NumPy, Cython и `rybind11` там импортируются и работают. Cython и `rybind11` укладываются в 4 % от своих времён на CPython на всех трёх вычислительных ядрах, а NumPy — на двух из трёх (сумма массива у Cython 1.025 против 1.011 мс, манделъброт у `rybind11` 0.774 против 0.752 мс, произведение матриц у NumPy 0.051 против 0.049 мс), и Cython на PyPy даже *быстрее*, чем на CPython, на двух ветвистых ядрах ($1.04\times$ на токенизации, $1.32\times$ на BFS). Дальше идут два обвала, и оба одной формы. Ядро бинарных деревьев на Cython занимает 493.8 мс на PyPy против 25.9 мс на CPython — $0.05\times$, — и это единственное ядро, чей `cdef class` выделяет $2.6 \cdot 10^5$ объектов CPython на вызов, каждый из которых PyPy приходится эмулировать. Манделъброт на NumPy занимает 77.1 мс против 9.98 мс — $0.13\times$, — и это единственное ядро NumPy, которое представляет собой длинную цепочку мелких проходов с маской, а не один заход в компилированный код.

В-третьих, **то же правило локализует происходящее с FFI.** Прогон наших ядер на C и Rust через `ctypes` на PyPy не стоит ничего на пяти ядрах из шести ($0.96-1.07\times$ от CPython, в обе стороны) и катастрофичен на шестом: токенизация, передающая по 2 МБ bytes на вызов, занимает 118.5 мс через C и 118.5 мс через Rust на PyPy против 22.3 и 22.4 мс на CPython — $0.19\times$ у обоих. Там, где через границу не идёт ничего, кроме целых чисел и заранее выделенных буферов, штраф исчезает полностью. Значит, шесть ядер разделяют не то, как часто пересекается граница, а *что* через неё идёт: цена — в маршалинге аргумента, а не в самом вызове.

Чего это стоит. PyPy стартует медленнее и держит больше памяти: чистый запуск интерпретатора — 26.1 мс против 10.6 у CPython, импорт стандартного набора модулей — 95.4 против 66.3 мс, а максимальный RSS после того же набора — 90.5 МБ против 29.8. Для долгоживущего сервиса ничего из этого не видно, для короткоживущего процесса это и есть большая часть прогона. Есть и цена, которой на этом хосте не измерить вовсе. На момент измерения PyPy реализует 3.11, поэтому кодовой базе на более поздней версии языка он недоступен, а NumPy удалил поддержку PyPy в феврале 2026 [7], так что приведённые выше строки NumPy-на-PyPy привязаны к серии 2.4 и вперёд не переносятся. И то и другое — проверяемые факты о той версии, которую читатель поставит, и рекомендация, рассчитанная на длинную дистанцию, обязана учитывать их наравне с ценой запуска.

Вывод. Возможность, которую легко не включить в сравнение: для вычислительного и насыщенного вызовами кода на чистом Python, который должен остаться чистым Python, альтернативный рантайм стоит $3.2-42.8\times$ без единой правки исходника — наибольший из измеренных здесь множителей, не требующий трогать код. Ограничение у него есть, но более узкое, чем «PyPy и нативный код не совмещаются». Почти вся компилируемая экосистема там работает и стоит несколько процентов, порядок величины стоит трафик через границу C-API — двухмегабайтный буфер, маршалируемый на каждый вызов `ctypes`, или четверть миллиона эмулируемых объектов CPython на вызов внутри расширения на Cython. Значит, две рекомендации — «перейти на PyPy» и «вынести горячий цикл за FFI» — складывать можно, если границу пересекают редко и дёшево, случай, который делает их взаимоисключающими, — вызов на каждую запись с крупной полезной нагрузкой. Numba — единственная технология, у которой пути нет вовсе: сборки под PyPy у неё не существует.

12 Обсуждение: таблица решений вместо рейтинга

Организовать рекомендации в этой области проще всего по технологиям — «здесь Numba, там C». Измерения говорят, что технология — неверная *первая* ось, и причины этому повторяются во всех восьми вопросах. Работающий индекс — это то, что делает нагрузка, а технология и её цена стоят в клетках.

1. На вычислительной работе технологии взаимозаменяемы, там, где доминируют аллокации, — нет. На скалярных вычислительных циклах Cython, Numba, C за двумя разными FFI и Rust совпадают на редукции с точностью до 0.2%, а там, где не совпадают, расходятся на 18% на одном ядре и в 1.8 раза на другом, в обе стороны и без постоянного победителя — против тех $32-128\times$, которых стоит сама компиляция (§4). Измерения поэтому между ними не выбирают, а то, чем этот выбор решается — сложность сборки, отлаживаемость, владение кодом, — лежит за пределами измеряемого здесь. Что *действительно* различает результаты, так это класс, к которому относится нагрузка: те же четыре технологии охватывают $52-61\times$ на вычислительных ядрах и $7.3-13.6\times$ на ветвистых и с аллокациями (§5), — а внутри второго класса они и между собой перестают быть взаимозаменяемыми, расходясь в $3.7-5.3$ раза ровно там, где выделяются мелкие объекты.

2. Доминируют решения о том, что программе разрешено делать, а не о том, каким инструментом это делается. Решение писать горячий цикл на языке, который вообще можно скомпилировать, стоит больше,

чем любой выбор, сделанный внутри этого языка, — кроме случая, когда пределом становится модель объектов: там обе величины одного порядка, потому что компиляция двоичных деревьев вообще даёт Cython 4.9×, а выбор реализации, владеющей собственной памятью, добавляет к этому ещё 3.5–5.3 раза (§5), — а распрямление структуры данных без компиляции цикла сделало наш обход графа на 12% *медленнее*. Отложить исполнение тел модулей стоит четырёх порядков задержки импорта (§7) и меняет момент, когда всплывают ошибки импорта. Разрешить переассоциацию операций с плавающей точкой стоит 2.42× на редукции, и это меняет результаты — и никакого флага компилятора для этого не нужно, поскольку перегруппировка накопления в исходнике стоит 2.74× при неизменных флагах (§4.1). Каждое из этих решений — о том, что программе разрешено делать, и каждое доминирует над выбором инструмента, сделанным под ним. Флаги ниже этих решений их не заменяют: переход с -O2 на -O3 не стоил ничего измеримого на двух из трёх наших ядер и 0.1% на третьем, тогда как -march=native, единственный флаг, который порожаемый код действительно менял везде, сделал одно ядро в 1.25 раза быстрее, а другое — в 1.80 раза *медленнее* (§4.1).

3. У каждого выигрыша есть цена, и здесь она измерена рядом с ним. JIT-компиляция стоила одной наблюдаемой задержки в 215 мс, а это 7 вызовов или 7335 вызовов работы в зависимости от размера входа (§4.2). Отказ от GIL стоит 1.05–1.14× однопоточного замедления (§8). Static Python стоит переписывания на примитивных типах и оказывается в 10.2 раза *медленнее*, если не включить ещё и JIT (§7). FFI стоит 59–166 нс за пересечение против 31 нс у вызова на чистом Python: бесплатно для ядра на 10 мс и настоящая цена на каждую запись (§5).

Таблица 8 — это результат: классы нагрузок из §1 с измеренным эффектом и ценой в каждой строке. Именно в таком виде мы её и предлагаем.

Четыре вопроса к любой цифре о производительности. Работа над восемью исследовательскими вопросами этой статьи дала короткий чек-лист. Каков был размер входа — точка окупаемости смещается у нас на три порядка (§4.2)? Была ли базовой линией та же программа — поставляемый BLAS против наивного тройного цикла сравнением языков не является (§4)? Менялась ли одна переменная или две — опции сборки интерпретатора стоят нескольких выпусков работы над интерпретатором, поэтому серия прогонов по версиям на поразному собранных бинарниках не измеряет ни того, ни другого (§6, §9)? И чего это стоило — на коде, который приём не ускоряет, на границе, которую он вводит, в семантике, которую он ослабляет (§8, §5, §4.1)? Цифра, выдержавшая эти четыре вопроса, годится для действия.

13 Ограничения достоверности

Одна машина — и что это не разрешает. Каждое измерение здесь получено на одном ноутбуке x86-64. Абсолютные времена — свойства этой машины; переносимы отношения, да и те лишь настолько, насколько переносим стоящий за ними механизм. Свидетельство на уровне машинного кода из §4.1 — самый ясный случай: то, какие флаги порождают одинаковый код, есть свойство этого компилятора и этой цели, и мы можем показать это прямо, а не предупреждать об этом отвлечённо. Поэтому выносить отсюда надо не рекомендацию про флаги, а то, что действие флага на порождаемый код приходится устанавливать заново для каждого компилятора и каждой цели, а дизассемблирование решает этот вопрос за секунды.

Привязка к ядрам убирает не весь разброс. Привязка однопоточных наборов к производительному классу (§3) убирает крупнейший источник необъяснённого разброса, но не весь: с этих ядер ничего не *вытесняется*, поэтому операционная система и любая другая работа на машине по-прежнему могут быть на них запланированы. Настоящая изоляция потребовала бы резервирования ядер на загрузке, чего мы не делали. Исследование масштабирования по потокам из §8 производительным классом ограничиться не может — оно доходит до восьми потоков, а в этом классе шесть ядер, — поэтому оно идёт на процессорах 0–13: это шесть производительных ядер и восемь из десяти энергоэффективных, перечисленных в таблице 1. Кривая приводится целиком, а не сводится к одному « $N \times$ »: после шести потоков добавленные потоки попадают на более медленные ядра.

Шесть ядер не представляют все нагрузки Python. Они выбраны так, чтобы воплотить классы нагрузок из §1, и разделение «вычислительные / ветвистые» намеренно доведено до крайности, чтобы поведение технологий разошлось.

Чего машинная проба не видит. Машинная проба из §3 однопоточная, и предел её работа нашла на собственном опыте: во время одного прохода посторонняя работа на машине замедлила ограниченный памятью бенчмарк NumPy на 13–34%, тогда как проба доложила о хосте как о несдвинувшемся с точностью до 0.03%. Она не ошиблась — конкуренция за ядра, которыми однопоточный цикл не пользуется, по построению для неё

невидима. Поэтому параллельные и ограниченные пропускной способностью памяти бенчмарки несут остаточный риск, который проба не покрывает, а затронутый набор был перемерен на простаивающей машине, а не приведён как есть.

Насколько машина сдвинулась, пока шли измерения. Машинная проба показывает, что хост движется мало: по 46 файлам результатов, которые её несут, самый медленный запуск составляет 1.014 от самого быстрого, а девяностый процентиль — 1.009. Это измеренное утверждение, а не предположение, и проба ровно для него и заведена. Сравнения *внутри* одного запуска набора подвержены дрейфу меньше всего: набор идёт минуты, и его бенчмарки соседствуют во времени. Сравнения *между* запусками подвержены больше всего, и серия прогонов по версиям имеет ровно такую форму — шесть отдельных прогонов одного набора, запущенных последовательно, с межверсионными отношениями, снятыми поперёк них. Машинная проба это ограничивает: это один и тот же бенчмарк во всех шести, поэтому его разброс по шести прогонам — прямое измерение того, насколько сдвинулся хост, пока их собирали, и он приводится вместе с серией. Там, где утверждение держится на величине меньше этого разброса, оно приводится вместе с этим разбросом (§3), там, где оно держится на множителе, дрейф не является определяющим источником неопределённости. Остаётся разрешающая способность, указанная в §3: её хватает для тех множителей, на которых держатся выводы работы, и не хватает для нескольких процентов, поэтому те немногие утверждения, которые держатся на нескольких процентах, несут вердикт значимости.

Происхождение интерпретаторов. То, что все интерпретаторы собраны здесь (§6), — не только контроль, ради которого это делалось, но и ограничение: работа измеряет то, что делают эти исходники, собранные таким образом, а интерпретатор, полученный иначе, — другой бинарник, поведение которого здесь не установлено.

14 Воспроизводимость

Артефакт, сопровождающий работу, содержит измерительный слой, все ядра на всех языках, скрипты сборки интерпретаторов, сырые файлы результатов ruperf и генераторы графиков и таблиц. Полная последовательность:

```
bash bench/host_topology.sh           # классы ядер -> привязка + results/host.json
OUT=<dir> bash bench/build_uniform.sh  # каждый интерпретатор: один компилятор, одна
                                      # строка configure, 3.10-3.14 + free-threaded
UNIFORM=<dir> bash bench/setup_env.sh  # venv на этих интерпретаторах; build_all.sh
                                      # собирает в них, поэтому идёт первым
CODON_DIR=<dir> PYPY=<py>py>/bin/py3 \
  bash bench/build_all.sh             # Cython, pybind11, Codon, C (6 наборов флагов +
                                      # лестница накопителей), Rust
OUT=<dir> LLVM19=<llvm19>/bin \
  bash bench/b5_buildflags/build_real.sh # семь конфигураций сборки; для tier-2 JIT
                                      # нужен LLVM 19
REAL=<dir> bash bench/setup_env.sh      # и ещё раз, для этих семи: они собраны без
                                      # pip, поэтому этот проход идёт после
                                      # build_real.sh
bash bench/b2_branchy/codegen_check.sh # results/codegen/summary.json для таблицы
                                      # со счётчиками инструкций C против Rust
UNIFORM=<dir> REAL=<dir> PYPY=<py>py>/bin/py3 bash bench/run_campaign.sh \
  compute branchy versions spec threads builds pipeline pypy
OUT=/path/cinder bash bench/b6_cinderx/build_cinderx.sh
CINDER=/path/cinder bash bench/b6_cinderx/run_b6_native.sh  # обе конфигурации CinderX
CINDER=/path/cinder bash bench/b6_cinderx/run_jitopts.sh    # абляция опций JIT
<venv>/bin/python bench/b6_cinderx/run_gc_scale.py --label ... # развёртка по сборщику
<venv>/bin/python bench/plots/make_figures.py                # figures/      (английские подписи)
<venv>/bin/python bench/plots/make_tables.py
<venv>/bin/python bench/plots/make_figures_ru.py            # figures-ru/  (те же данные, русские подписи)
<venv>/bin/python bench/plots/make_tables_ru.py
<venv>/bin/python bench/plots/check_overlaps.py              # проверка вёрстки; должна вернуть 0
<venv>/bin/python bench/plots/check_overlaps.py --ru         # то же для русских рисунков
<venv>/bin/python bench/plots/summarize.py                   # каждое число, приведённое в тексте
```

Каждый график и каждая таблица пишутся этими генераторами из results/ruperf/, и каждая цифра прозы выведена из тех же файлов, а не записана по ходу прогона: summarize.py печатает медианы и отношения рядом с записями, из которых они взяты, а вердикты значимости — это ruperf compare_to по двум наборам значений.

Файл результатов — это собственный формат `pyperf`, поэтому его можно прочитать и через `pyperf stats` и `pyperf compare_to` без нашего кода. Каждый бенчмарк несёт в метаданных свой набор, ядро, реализацию, параметры и метку конфигурации, а каждый набор пишет боковой файл с вердиктами корректности, вариантами, которые не удалось собрать, описанием хоста и структурными наблюдениями, которые не являются замерами, — чтобы то, чего `pyperf` не измерял, нельзя было принять за то, что он измерял.

15 Заключение

Эта работа ставила целью измерить арсенал приёмов ускорения Python в условиях, которые позволяют их сравнивать: восемь вопросов, один протокол измерения, одно происхождение интерпретаторов, один документированный хост и код за каждой цифрой. Получается из этого таблица, индексированная нагрузкой, а не инструментом, а разделы ниже — основные выводы, которые из неё следуют.

Что даёт технология. Компилированный код — любым из проверенных путей — на один-два порядка быстрее интерпретатора CPython на вычислительных ядрах (31–32× на сумме массива, 52–62× на сетке манделъброта, 46–128× на наивном произведении матриц). Каким именно путём идти, значит гораздо меньше, чем то, что идти надо хоть каким-то: на скалярной редукции они совпадают с точностью до 4.4%, а на двух других ядрах расходятся на 21% и в 2.8 раза, в обе стороны и без постоянного победителя. На фоне множителя от тридцати до сотни скорость между Cython, Codon, Numba, C и Rust не выбирает, а то, чем этот выбор решается — сложность сборки, отлаживаемость, владение кодом, — эта работа не измеряет. Разброс при этом настоящий, а не равенство, и проекту, вся стоимость которого сводится к одному ядру, стоит измерить своё. Отказ от GIL даёт 5.60× на арифметике и 6.05× на насыщенном ветвлениями коде на восьми потоках, а контроль `PYTHON_GIL=1` на том же бинарнике — который вместо этого теряет 14–20% на восьми потоках — относит это масштабирование к блокировке, а не к сборке: цена его получения устроена ровно наоборот, и о ней ниже.

Где сработал не тот механизм, что стоит в названии приёма. Крупнейшие рычаги в вычислительной части — это решение компилировать вообще и, там, где операция уже есть в библиотеке, поставляемый BLAS, флаги же под этими решениями ведут себя менее единообразно, чем звучат их имена. Дизассемблирование каждого варианта флагов и решает, что каждый из них на самом деле меняет: `-O2` и `-O3` порождают побайтово одинаковый код на двух ядрах из трёх и расходятся на одну инструкцию на третьем, и времена этому соответствуют, так что переход между ними здесь не даёт ничего. Другой код везде порождает только `-O3 -march=native`, и именно его эффект направлен в обе стороны (§4.1). Отключение векторизатора под этим флагом не изменило ни порождаемого кода, ни времени ни на одном ядре, а SIMD-инструкции появляются только после того, как разрешён `fast-math`, — так что архитектурный флаг оплачивает здесь не векторизацию. Контракт арифметики с плавающей точкой стоит 2.42× на редукции и тоже, по сути, не является флагом: четыре независимых накопителя, выписанных в исходнике на C при обычном `-O2` и без всякого `fast-math`, на 13% быстрее самого `-ffast-math`, а восемь накопителей хуже четырёх. Строка, которая читалась бы как «использовать нативные флаги», читается вместо этого как «определить, что арифметике разрешено делать, знать, что вы это определяете, и дизассемблировать результат на той цели, которую отгружаете». На нерегулярном коде компилятор побеждает не ветвление — Cython и C расходятся на нашем обходе графа на полпроцента, а нативные пути опережают не более чем на 17% на разборе байтов, — а выделение памяти, и именно там C и Rust уходят вперёд в 3.7–5.3 раза.

Где ответом является распределение. Прогресс интерпретатора от 3.10 к 3.14 реален и неравномерен: ступеньку дала 3.11 (чтение атрибута 1.84×, вызовы методов 1.76× относительно 3.10), 3.12 часть этого отдала назад — она медленнее 3.11 на 14 из наших 16 операций, все 14 значимо, хотя именно в ней `none` стал бесмертным, — 3.13 изменила мало, а 3.14 восстановилась до 1.97× и 2.04×, тогда как построение dict-литерала на 3.14 идёт со скоростью 0.74× от 3.10. На уровне сборки полезной записью оказывается комбинация, а не какая-либо одиночная опция: PGO сам по себе стоит 1.02×, PGO вместе с `full LTO` — 1.10×, а добавление к этому `-march=native` — 1.12×, единственная операция, которая проигрывает во всех шести конфигурациях, — запись атрибута, на 0.88–0.98×. CinderX так же сопротивляется однословному вердикту. Он собирается в обеих своих конфигурациях с нулём ошибок компиляции, его JIT принудительно скомпилировал все восемь переданных ему функций без единого отказа, и все ядра прошли проверку корректности. Дальше его рантайм стоит 1–33% на шести наших нетипизированных ядрах в любой из конфигураций, а значит, цена лежит в пути исполнения, а не в флаге сборки, и JIT возвращает её на всех шести, исполняя их за 0.33–0.79× обычной сборки, — если позволить ему компилировать после специализации интерпретатором, а не до неё. Облегчённые фреймы сдвинули наши пробы менее чем на 3% на голом форке. Параллельный сборщик сдвинул свою пробу не в ту сторону, пока долгоживущая куча воркера была ему видна (в 1.48 раза медленнее), и в нужную — лишь после того, как `immortalize_heap()` убрал её из поля зрения (в 1.06 раза быстрее), так что переменной оказывается состав кучи, а не число потоков. Его единственный крупный выигрыш (Static Python, 11.4× поверх JIT) достигим только через последовательность инициализации, описанную лишь в руководстве, и только для переписанного типизированного кода. Измерение адаптивной конфигурации само оказалось работой: специализирующий путь для статических опкодов перенесли на 3.14 без обработчиков, в которые он специализируется, поэтому

Static Python падал на первой же специализации, пока мы их не дописали, — а дописанный, флаг снимает 43% с интерпретируемого Static Python и ровно ничего с компилируемого, что и есть самое ясное во всей работе высказывание о том, что примитивные опкоды — промежуточное представление для JIT, а не способ быстрее интерпретировать.

Цены, без которых множитель не превращается в решение. Числа, которые решают выбор и редко сообщаются: точка, в которой JIT окупается, — она смещается с 7 вызовов до 7335 в зависимости только от размера входа, против одной наблюдаемой задержки компиляции в 215 мс, цена каждой границы FFI — 59–166 нс за пересечение (§5), однопоточная цена отказа от GIL — 1.05–1.14×: её берёт сама free-threaded сборка, снята блокировка или нет, а возвращает второй поток. И цена Static Python: в 10.2 раза медленнее Python с объектами, если не включить ещё и его JIT, и всё ещё в 5.8 раза медленнее со включённым специализирующим путём интерпретатора. Сюда же относятся ещё два результата, потому что в них решает цена. PyPy даёт 3.2–42.8× на неизменённом коде на чистом Python — наибольший множитель, доступный без правки исходника. Но единственное ядро, чей вызов `ctypes` передаёт ему двухмегабайтный буфер, стоит там в 5.3 раза дороже, чем на CPython (118.5 против 22.3 мс), тогда как вызовы, передающие только целые числа, укладываются в 7%: «сменить рантайм» и «вынести горячий цикл в C» могут исключать друг друга. Второй результат — собственный экспериментальный JIT CPython: предложение на 1.04× в целом, которое тоже не бесплатно. Он улучшает 10 из наших 16 операций и замедляет 6, а целочисленную арифметику и запись атрибута — до 0.88–0.89×.

О методе. Три наших собственных результата были неверны, пока их не поймал протокол (§3), — ради этого протокол и нужен. Поймавшие их средства дешёвы: `pyperf` вместо самописного таймера, попеременное измерение там, где утверждение держится на нескольких процентах, проверка корректности в том же процессе, который будет измерять, и проба, измеряющая хост, а не программу.

Практически принцип композиции сохраняется с приложенным к нему более точным правилом: на нашем смешанном конвейере Numba на стадии, стоящей 3.6% прогона, даёт 1.01×, перенос стадии классификации — половины времени прогона — в C даёт 1.35×, а распределение работы поверх этого по четырём потокам даёт 0.72× под GIL и 1.26–1.28× на free-threaded сборке — всё равно ниже тех 1.35×, которые даёт одно хорошо нацеленное изменение. Композиция не аддитивна — она направляется профилем, и наложение сверх профиля забирает производительность назад. Решения об оптимизации следует индексировать тем, что делает нагрузка — цикл по массивам, выделение объектов, обход графа, запуск, распределение по ядрам, — а не именем инструмента: внутри класса нагрузки разброс между инструментами оказался мал рядом с решением воспользоваться хоть одним, а между классами один и тот же инструмент различался на порядок. Таблица 8 и есть этот индекс, и каждая цифра в ней воспроизводится из прилагаемого артефакта двумя командами.

Список литературы

- [1] V. Stinner. *pyperf: toolkit to write, run and analyze benchmarks*. Версия 2.10.0. <https://github.com/psf/pyperf>.
- [2] Python Software Foundation. *pyperformance: the Python benchmark suite*. <https://github.com/python/pyperformance>.
- [3] Meta Platforms. *Cinder: форк CPython от Meta, ориентированный на производительность, и CinderX — пакет расширений, несущий его JIT, Static Python, параллельный сборщик и ленивые импорты*. <https://github.com/facebookincubator/MetaPython> и <https://github.com/facebookincubator/cinderx>.
- [4] S. Gross. *PEP 703 — Making the global interpreter lock optional in CPython*. 2023.
- [5] M. Shannon. *PEP 659 — Specializing adaptive interpreter*. 2021.
- [6] The PyPy Team. *PyPy 3.11 (v7.3.20)*. <https://pypy.org>.
- [7] NumPy contributors. *MAINT: drop pypy — pull request #30764*, влит 3 февраля 2026. <https://github.com/numpy/numpy/pull/30764>.
- [8] S. K. Lam, A. Pitrou, S. Seibert. *Numba: a LLVM-based Python JIT compiler*. LLVM-HPC '15.
- [9] S. Behnel et al. *Cython: C-extensions for Python*. <https://cython.org>.
- [10] A. Shajii, G. Ramirez, H. Smajlović et al. *Codon: A Compiler for High-Performance Pythonic Applications and DSLs*. CC '23.
- [11] B. Bucher, S. Ostrowski. *PEP 744 — JIT compilation*. 2024. Draft.
- [12] E. Snow, E. Elizondo. *PEP 683 — Immortal objects, using a fixed refcount*. 2022.
- [13] B. Cannon, D. Viehland. *PEP 523 — Adding a frame evaluation API to CPython*. 2016.
- [14] The PyPy Team. *PyPy speed centre*. <https://speed.pypy.org>.

Таблица 8: Таблица решений по измерениям. Каждый множитель получен на единственном хосте измерений из таблицы 1, в колонке «цена» указано, что изменение приносит с собой.

нагрузка	что сработало	измерено	цена / оговорка
численный цикл по массиву ... и переассоциация допустима	любой компилируемый путь (Cython / Numba / C / Rust) fastmath / -ffast-math	32× ещё 2.42×	шаг сборки, четыре пути здесь лежат в пределах 0.2% друг от друга результаты меняются (здесь на $4.6 \cdot 10^{-5}$)
операция, которая уже есть в библиотеке	NumPy / поставляемый BLAS	до 1495.9×	только там, где операция существует, 4.7×, когда не подходит
ветвистый проход по плоским данным	Cython или Numba	12×	C и Rust впереди лишь на 8–17%
граф объектов с аллокациями	аллокатор, отличный от CPython: C или Rust за FFI либо Codon	17.2–20.8× против 3.9–4.9×	переписывание владения памятью или перекompиляция, если исходник берёт Codon
нерегулярный обход графа	компилировать его, одно плоское размещение выигрывает не даёт	9.9× (Cython и C расходятся на полпроцента)	распрямление на чистом Python дало 0.89×
объектный код на Python	обновиться до 3.11+	2.0–3.0× на атрибутах и методах	большую часть дала 3.11, 3.12 медленнее 3.11 на 14 из 16 операций, dict-литерал на 3.14 идёт со скоростью 0.74× от 3.10
CPU-нагрузка на Python по потокам	free-threaded сборка	5.6–6.1× на 8 потоках	1.05–1.14× однопоточной цены, которую берёт сборка, а не блокировка
параллельная работа, уже в C	потоки на любой сборке	5.8–7.2× на 8 потоках, сбалансированное ядро	GIL здесь ни при чём; несбалансированное даёт 1.8–3.1×
настройка сборки интерпретатора	PGO вместе с full LTO, а не PGO отдельно	1.10× (1.12× с -march=native)	один PGO даёт 1.02×, запись атрибута проигрывает во всех шести, 0.88–0.98×
собственный tier-2 JIT CPython	измерить его: он помогает большому числу операций, чем портит	1.04× в целом	6 операций из 16 медленнее (целочисленная арифметика и запись атрибута 0.88–0.89×), по-прежнему помечен экспериментальным
чистый Python, который должен остаться чистым	PyPy	3.2–42.8×	вызов ctypes, передающий буфер в 2 МБ, стоит там в 5.3 раза дороже, чем в CPython, Numba нет, поддержка экосистемы слабеет
редукция, ограниченная своей цепочкой зависимости	перегруппировать накопление в исходнике	2.74× при неизменных флагах, выше 2.42× у -ffast-math	другой порядок суммирования — независимо от того, флаг это разрешил или исходник (§4.1)
типизированный горячий код, готовы переписать	CinderX Static Python + JIT	11.4× поверх JIT	инициализация описана только в руководстве, без JIT в 10.2 раза медленнее и в 5.8 раза медленнее даже со специализирующим путём интерпретатора
запуск, в котором доминируют импорты	ленивые импорты Cinder	19.7 мс → менее 1 мс	только форк, ошибки импорта уезжают на первое обращение, первое обращение стоит 0.35 мс
смешанный конвейер	атаковать крупнейшую стадию и на этом остановиться	1.35× с одной стадией	Numba на стадии в 3.6% дала 1.01×, добавление потоков дало 0.72× (GIL) и 1.28× (free-threaded)